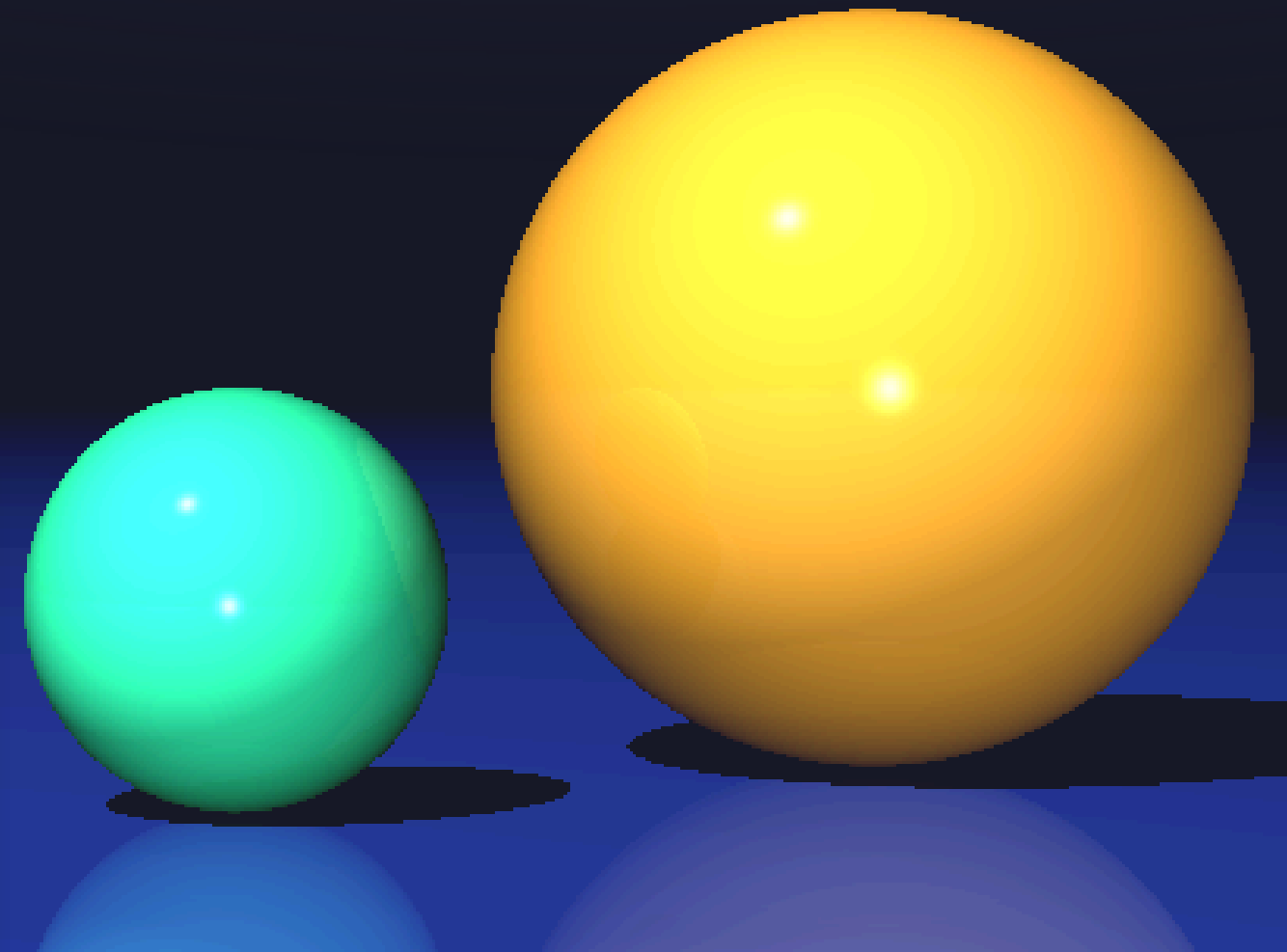


Assignment 2: Ray Tracing

Viewing rays, ray-primitive intersection,
Blinn-Phong shading, shadows and mirrors.



Contents

01 Setup

02 Rays and the camera

03 Part 1: Ray casting

04 Part 2: Ray tracing

05 Grading and debugging

Fri Oct 9

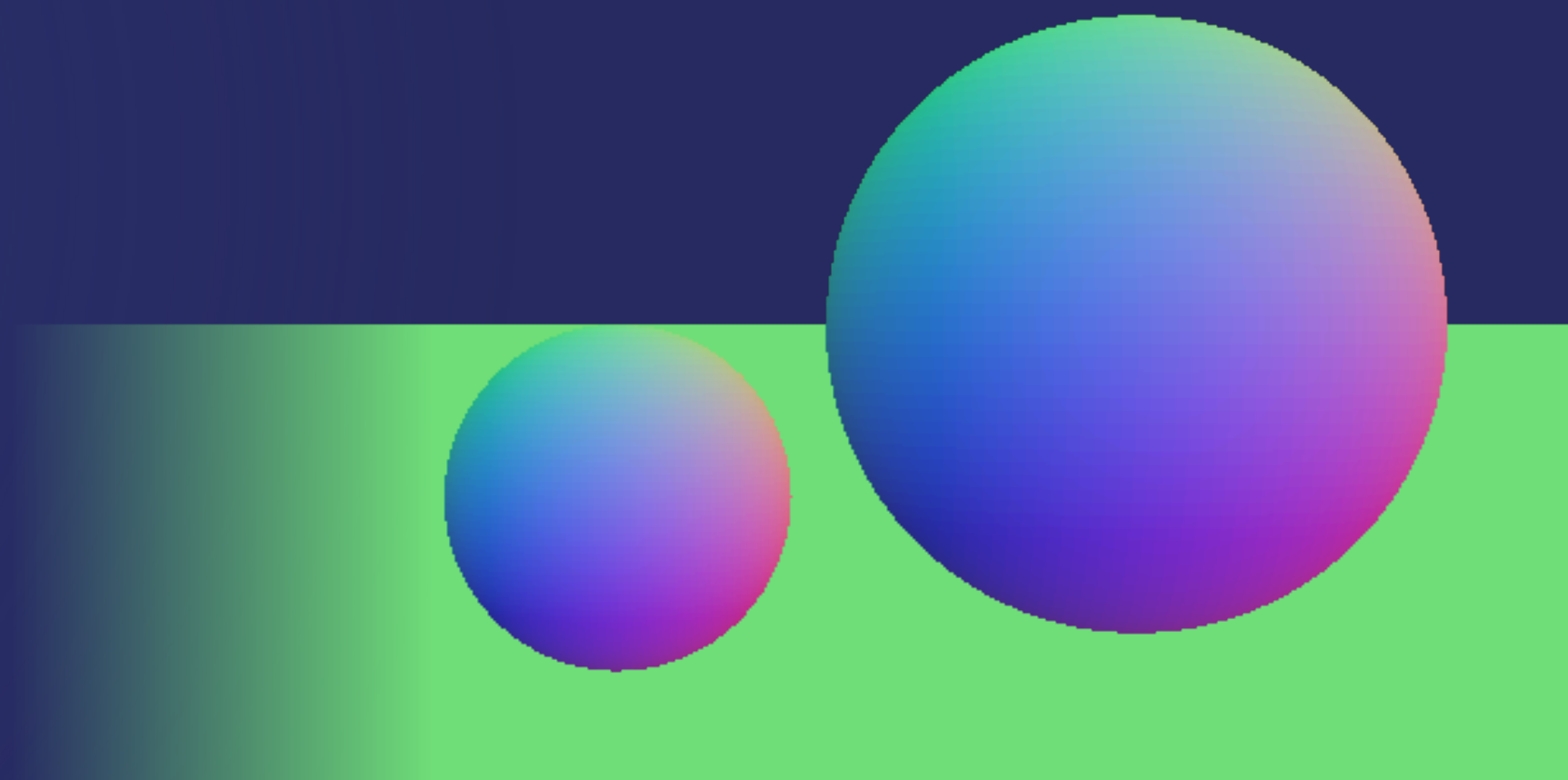
Assignment 2 is due at 11:59 pm. Assignment 1 is due first — Friday September 25. Grace tokens apply automatically, 12 hours each, as covered last week.

Where to get help

- Assignment questions go on the A2 repository's GitHub Issues page — answer each other there, but never post code.
- For everything else, email the TAs: csc317tas@cs.toronto.edu
- "Can you find the bug?" is for tutorials and office hours — not Issues or email.

01

— Setup



Getting set up

```
# 1. Clone the repository. Do not fork – forks are public.
$ git clone git@github.com:panuelosj/computer-graphics-csc317-A2-Ray_Tracing.git
$ cd computer-graphics-csc317-A2-Ray_Tracing
# 2. Once: create ./venv and install numpy, pillow, scipy, polyscope
$ ./init.sh # Windows PowerShell: .\init.ps1
# 3. In every new terminal: activate the environment
$ source venv/bin/activate # Windows: .\venv\Scripts\Activate.ps1
```

- Windows: if scripts are blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first.
- A1 and A2 each have their own venv. Activate the one for the repository you are in.

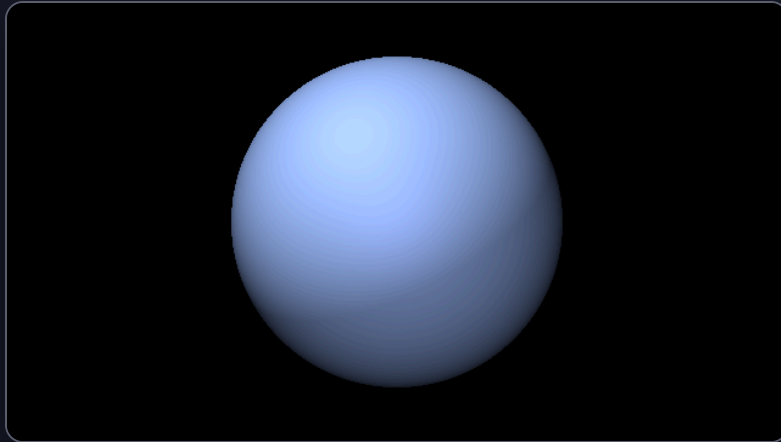
Repository layout

`src/` is the only directory you edit

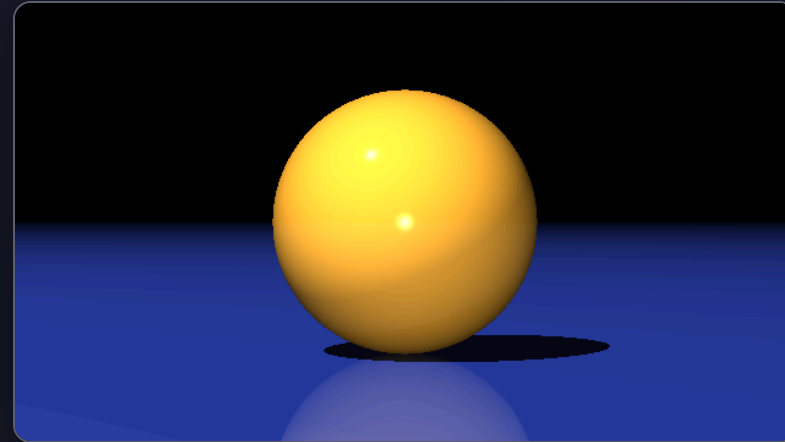
- Ten files, one function each. Every stub raises `NotImplementedError`; its docstring is the specification.
- Read `backend/scene.py` too — it defines `Ray`, `Camera`, `Material` and the geometry your functions receive.

<code>README.md</code>	tasks, math and grading
<code>init.sh / init.ps1</code>	one-time venv setup
<code>main.py</code>	renders a scene
<code>run_tests.py</code>	PASS/FAIL table
<code>check_my_work.py</code>	checks weighted by marks
<code>backend/</code>	scene + render loops
<code>cgcommon/</code>	helpers (do not edit)
<code>src/</code>	YOUR ten functions
<code>tests/</code>	checks (do not edit)
<code>data/</code>	scenes and meshes
<code>docs/images/</code>	reference renders

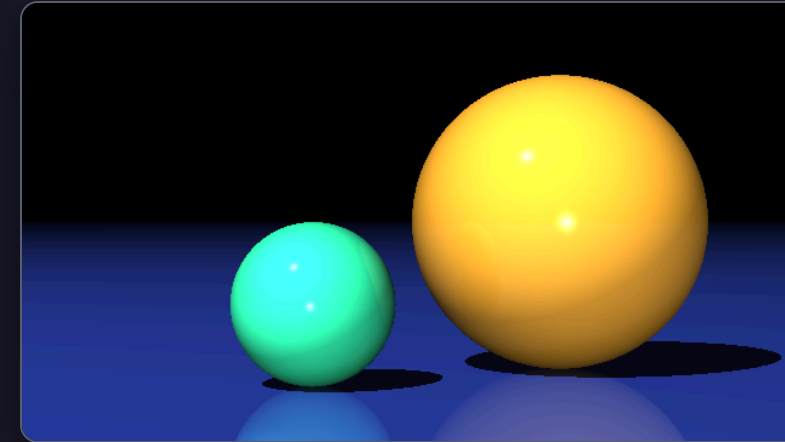
The scenes in data/



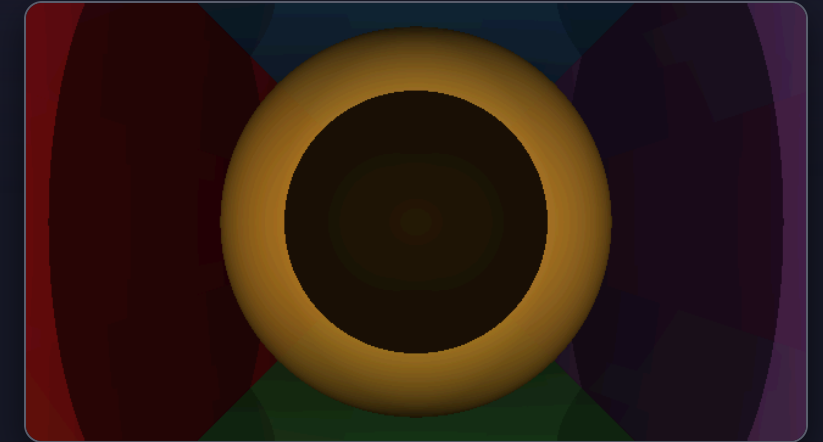
sphere — start here



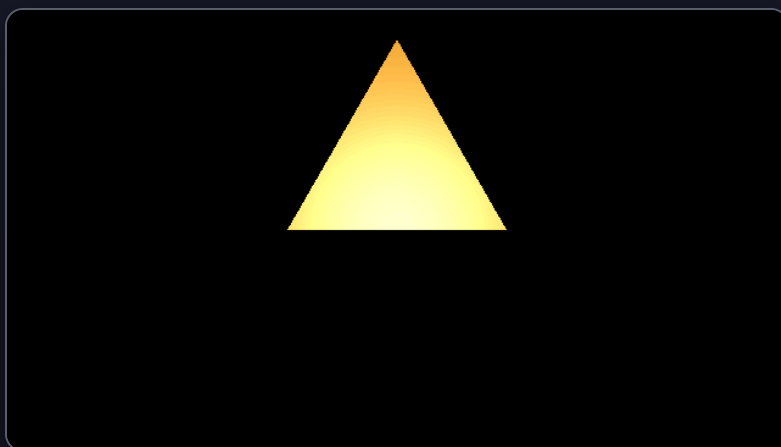
sphere-and-plane — our running example



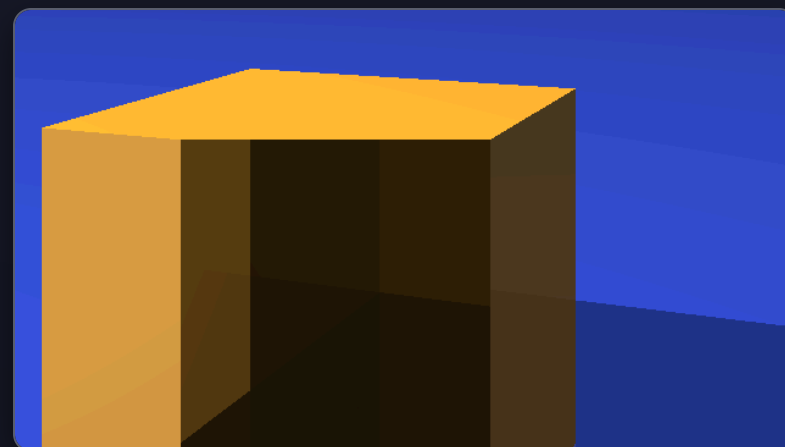
two-spheres-and-plane — the default



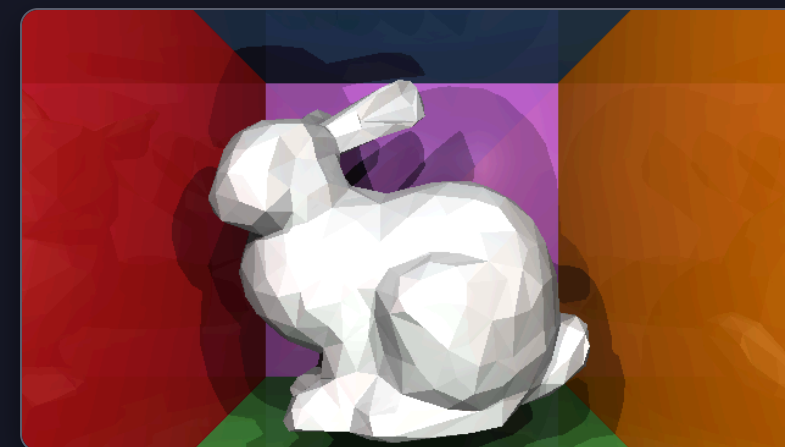
inside-a-sphere — the far root



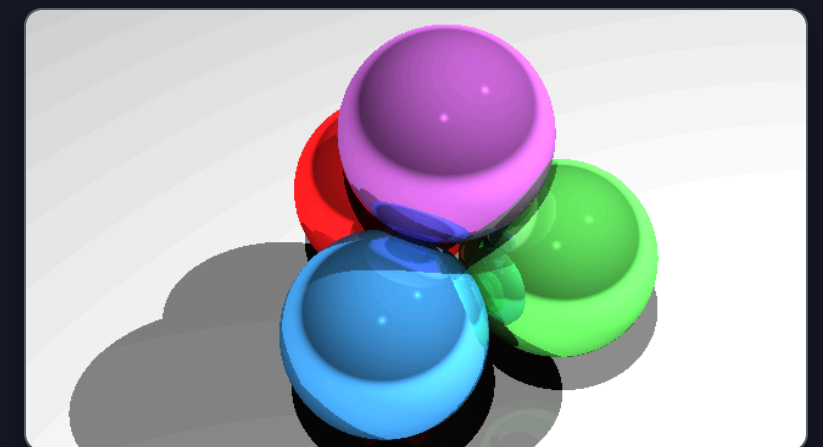
triangle



cube-and-plane — a small soup



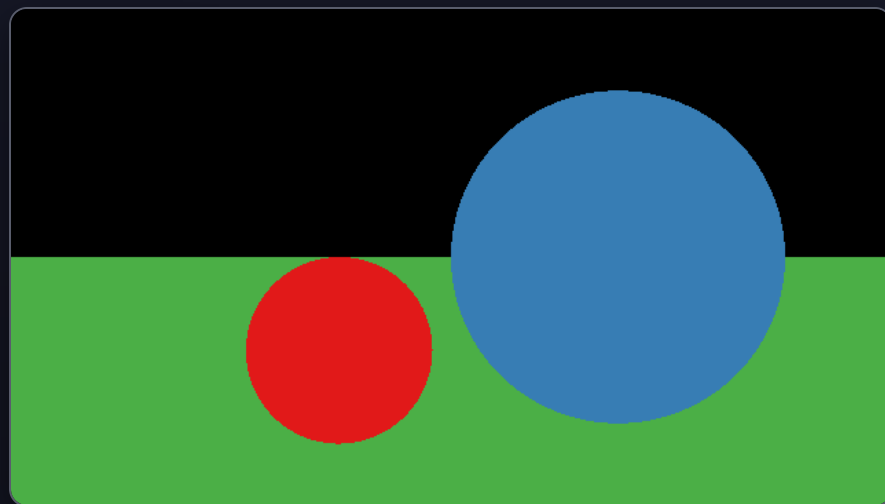
bunny — 1000 triangles



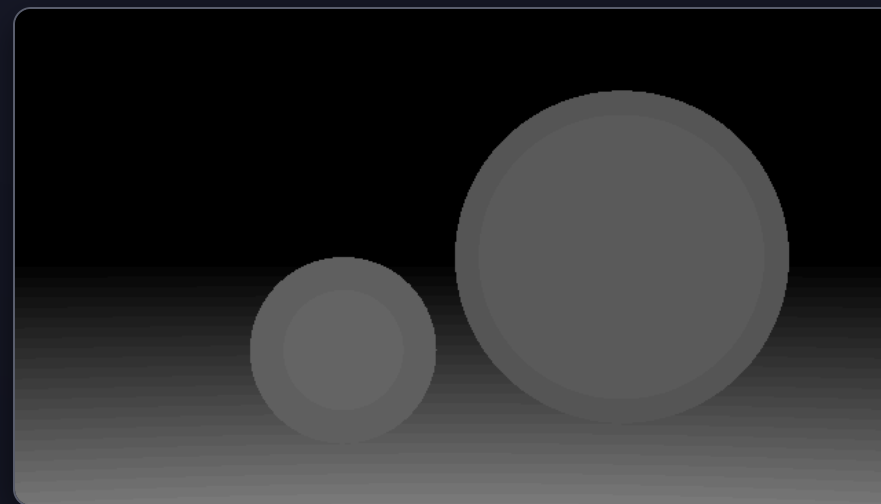
sphere-packing — mirrors facing mirrors

Running and checking your work

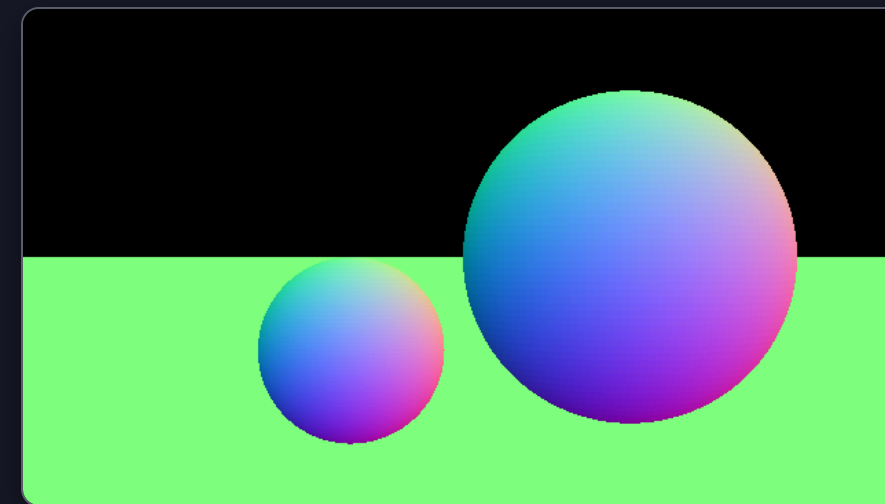
```
$ python main.py                                # default scene, 320×180 → out/  
$ python main.py --scene data/two-spheres-and-plane.json --height 360  
$ python main.py --part 1                        # only the ray-casting buffers ↓  
$ python run_tests.py                           # PASS/FAIL per function + "Validated X/10"  
$ python check_my_work.py                       # the same checks, reported as marks
```



`.id.png` — which object each ray hit



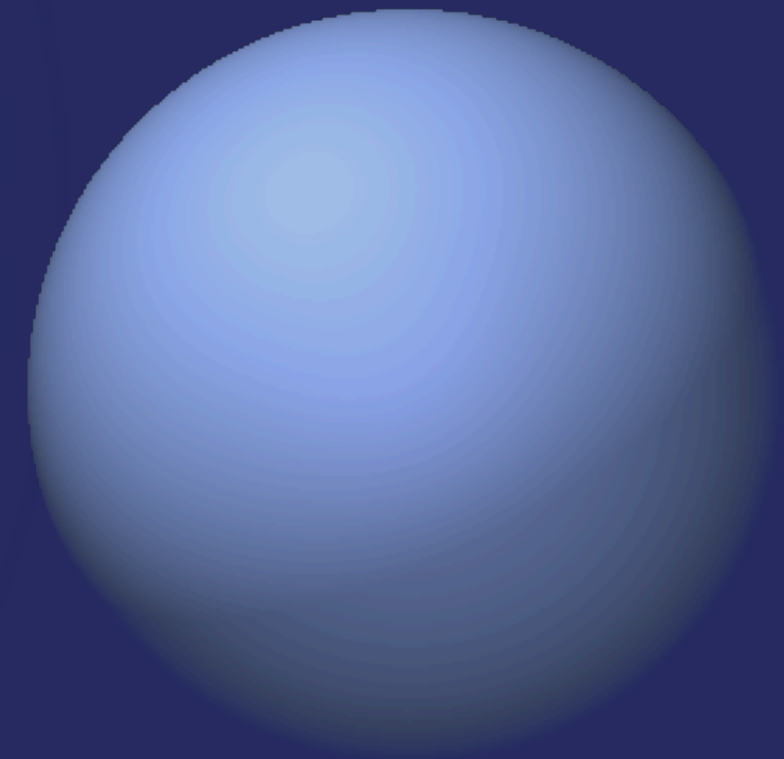
`.depth.png` — nearer is brighter



`.normal.png` — surface direction

02

— Rays and the camera



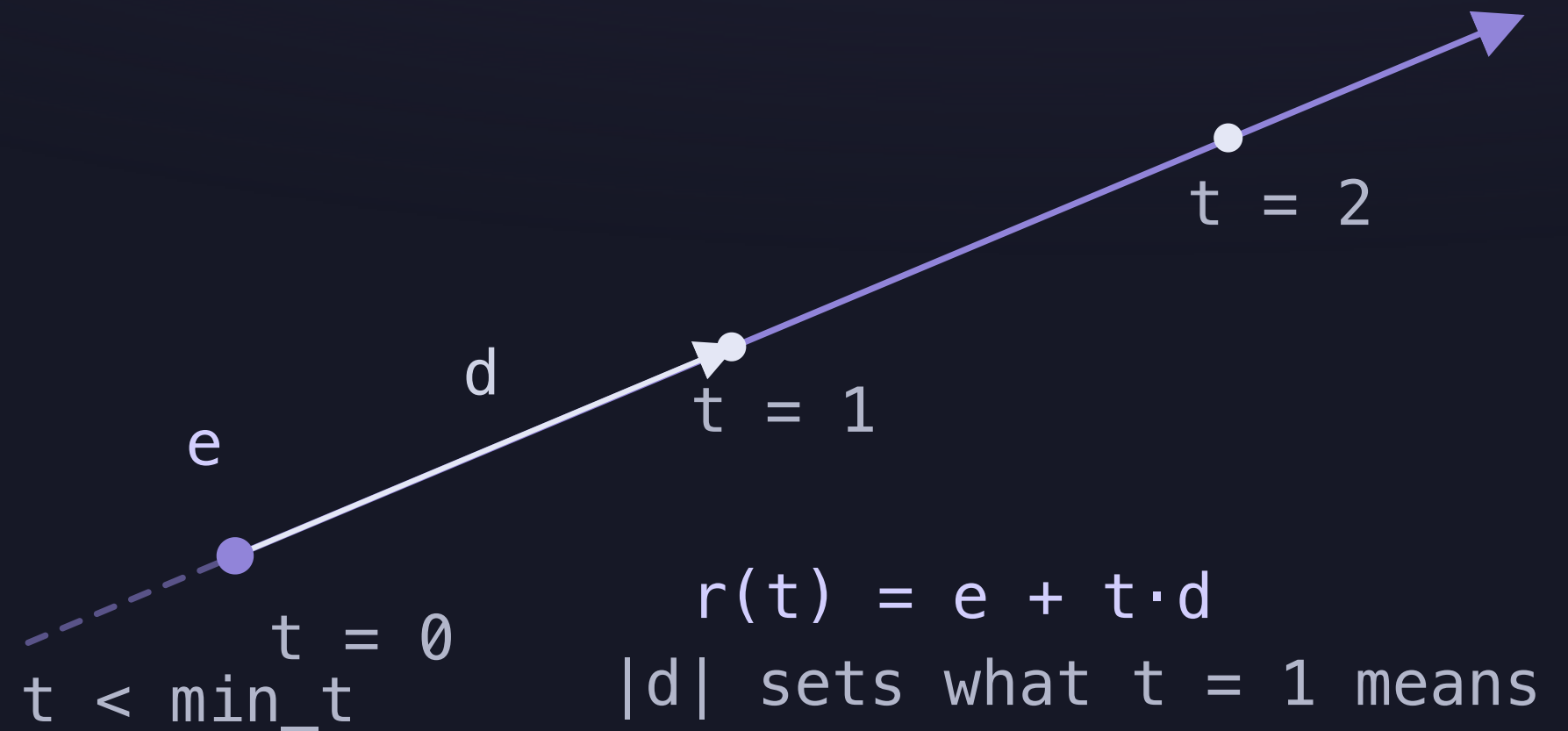
A ray is a point and a direction

ONE PARAMETER, t

- $r(t) = e + t \cdot d$. Each $t \geq 0$ is one point along the ray.
- **d is *not* normalised**: its length chooses what $t = 1$ means — used on purpose in three of the ten tasks.

BACKEND.SCENE.RAY — GIVEN TO YOU

```
@dataclass
class Ray:
    origin: np.ndarray      # e
    direction: np.ndarray  # d, NOT unit:
    # intersects return (hit, t, n)
```



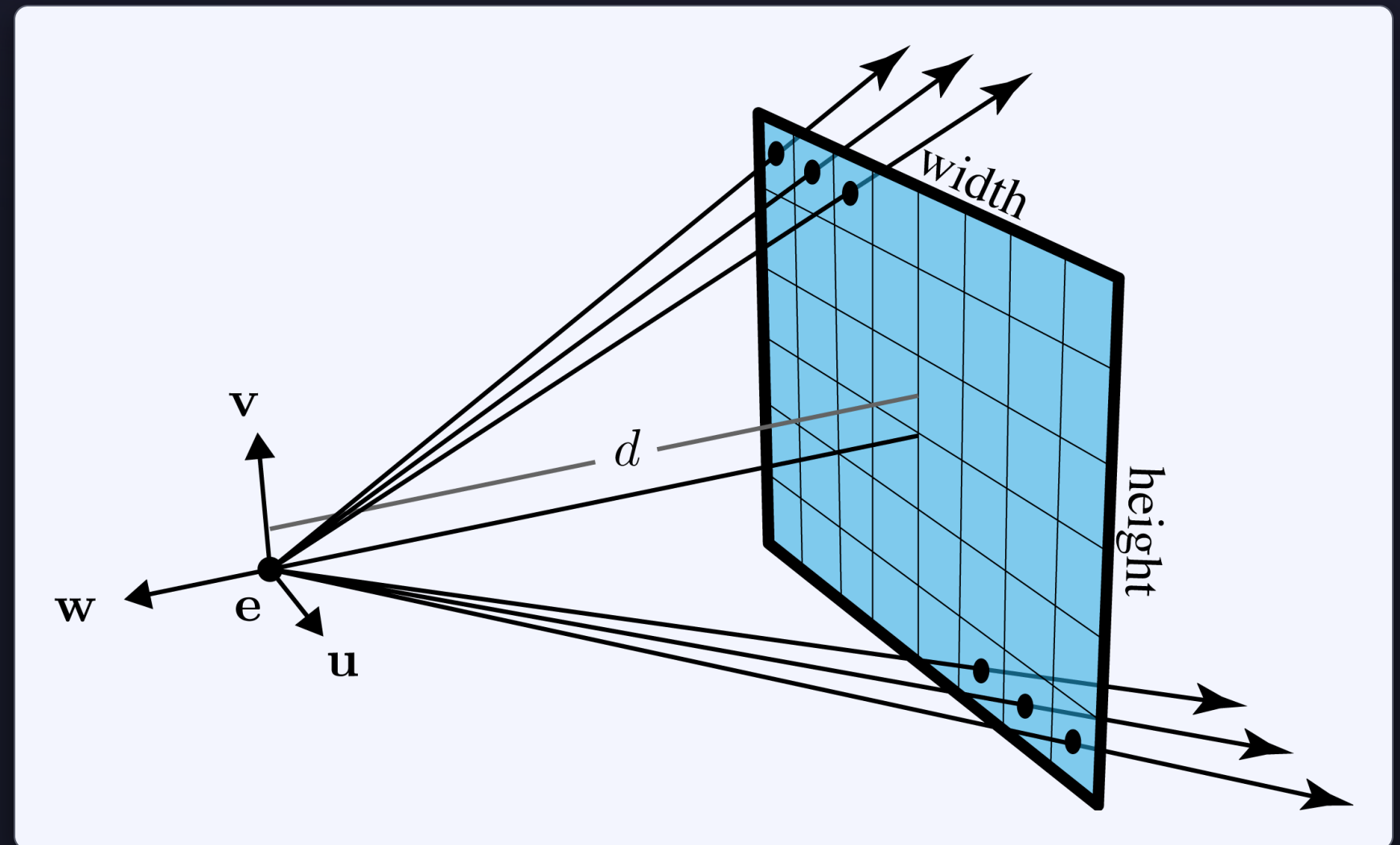
The camera

BACKEND.SCENE.CAMERA — GIVEN TO YOU

```
@dataclass
class Camera:
    eye: np.ndarray    # the point e
    u: np.ndarray      # right
    v: np.ndarray      # up
    w: np.ndarray      # -w looks in
    d: float           # focal length
    width: float        # plane size in
    height: float       # scene units
```

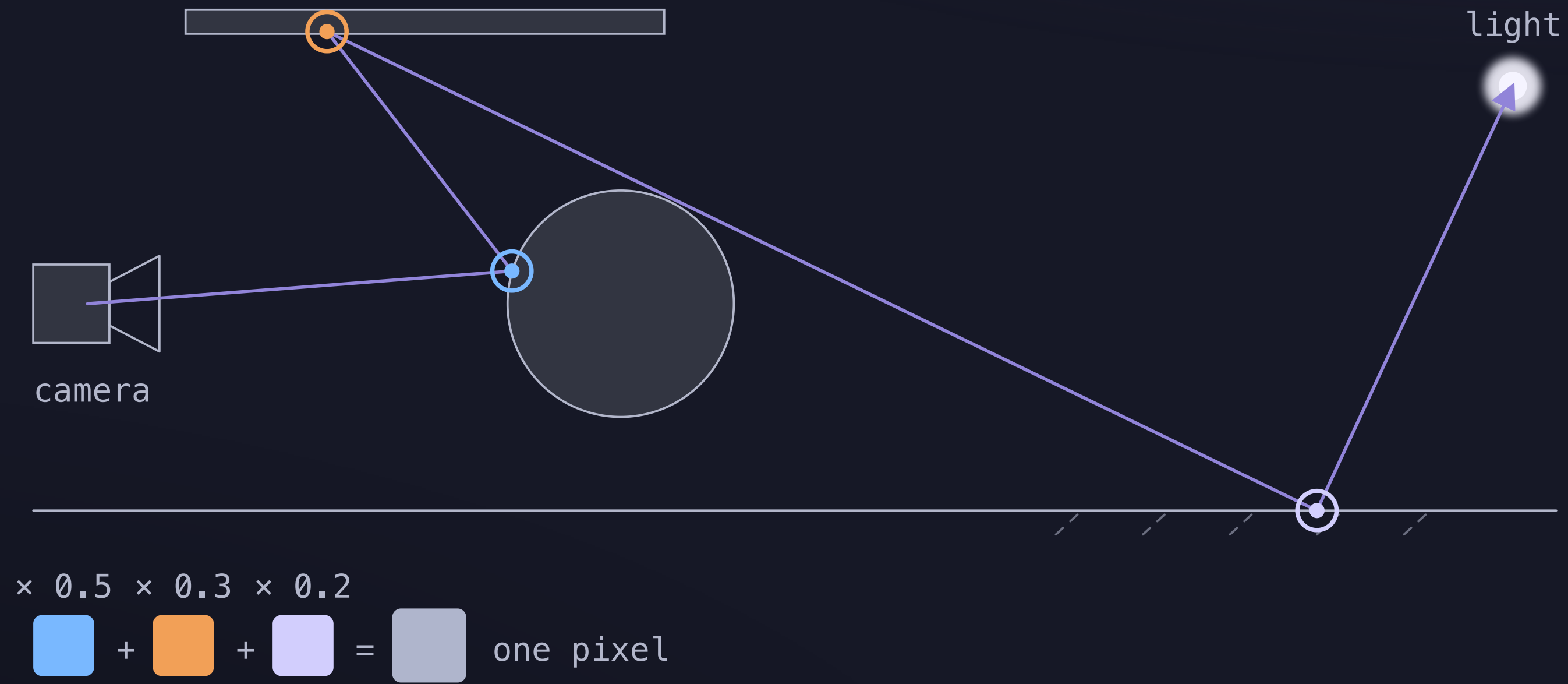
PIXEL CONVENTIONS

- i is the row (down), j the column (right), origin top-left
- as in A1. The textbook differs.



the pinhole camera: eye e , frame $u \ v \ w$, focal length d , image plane $width \times height$

The whole idea — one ray, gathering colour



Ten functions, 100 marks

PART 1 · RAY CASTING	MARKS	WHAT IT DOES	PART 2 · RAY TRACING	MARKS	WHAT IT DOES
viewing_ray.py	12	Ray through a pixel	reflect.py	6	Mirror a direction
sphere_intersect.py	12	Quadratic formula	point_light_direction.py	6	Toward a point light
plane_intersect.py	8	Implicit plane	directional_light_direction.py	6	Toward a far light
triangle_intersect.py	12	Cramer's rule	blinn_phong_shading.py	14	Shade with shadows
first_hit.py	10	Closest of all objects	raycolor.py	14	Shade + recurse

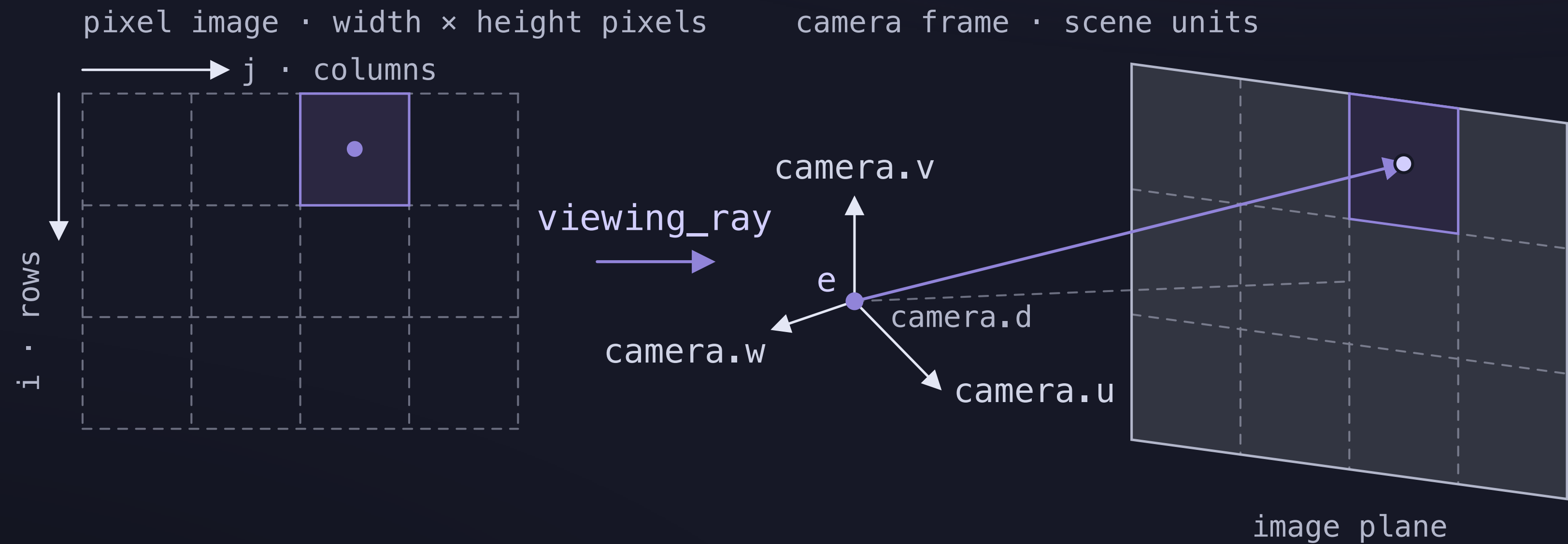
03

— Part 1: Ray casting



viewing_ray — from pixel indices to the image plane

CONCEPT



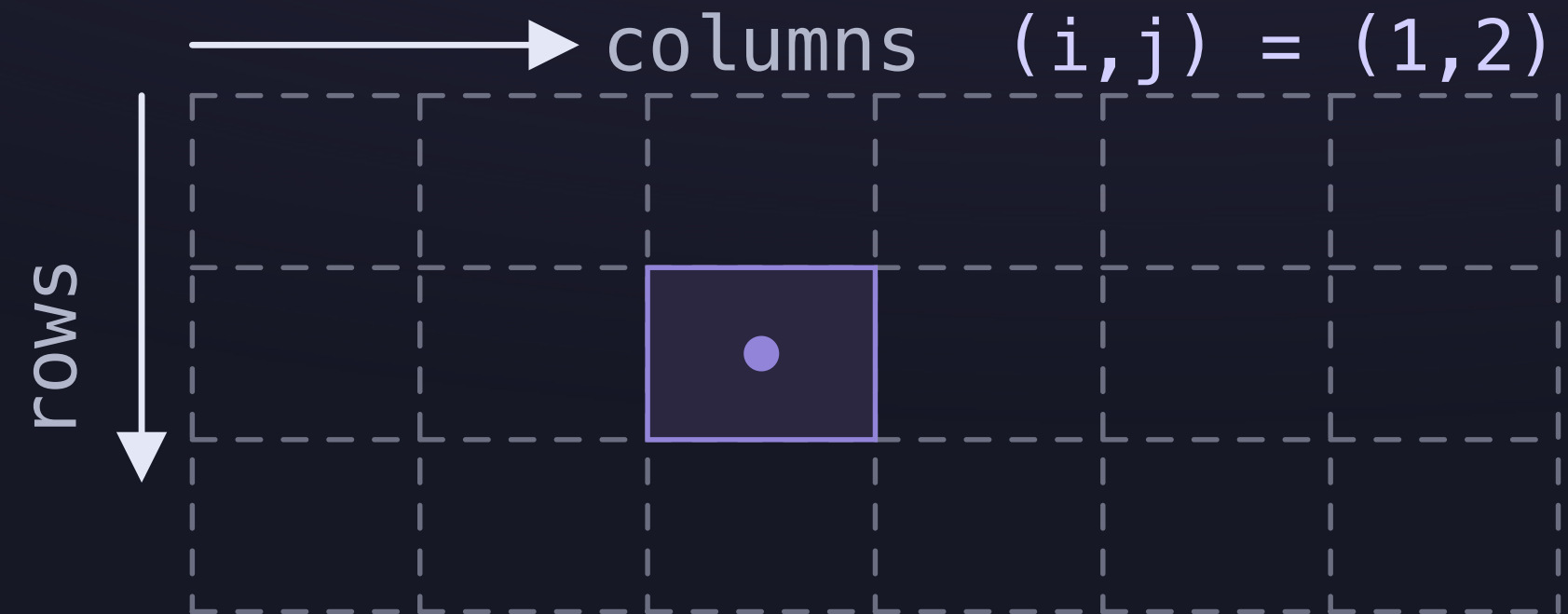
viewing_ray

YOUR TASK — 12 MARKS

- `viewing_ray(camera, i, j, width, height)` → a Ray through pixel (i, j) 's *centre*.

ALGORITHM

- 1 Pixel centre: $(j + 0.5, i + 0.5)$ — half a pixel in from the corner.
- 2 Scale into image coordinates on the plane, then negate v : rows grow down, `camera.v` points up.
- 3 $\text{Direction} = -\text{camera.d} \cdot \text{camera.w} + u \cdot \text{camera.u} + v \cdot \text{camera.v}$.
- 4 Return `Ray(camera.ey, direction)` — unnormalised, so $t = 1$ lands on the image plane.



$$\text{centre} = (j + 0.5, i + 0.5)$$

Row i counts **down**, column j counts right.

Forgetting the half pixel shifts the whole image and fails the test.

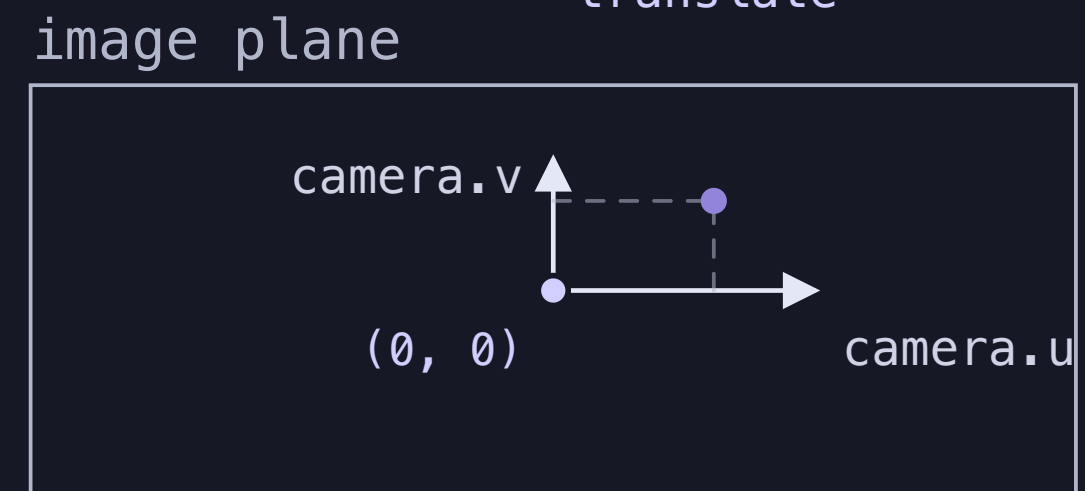
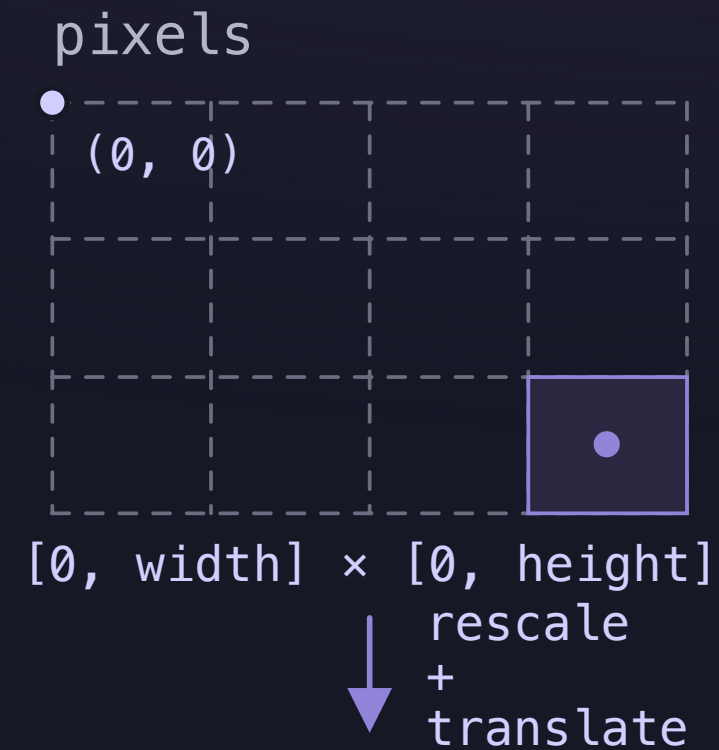
viewing_ray

YOUR TASK — 12 MARKS

- `viewing_ray(camera, i, j, width, height) → a Ray` through pixel (i, j) 's *centre*.

ALGORITHM

- 1 Pixel centre: $(j + 0.5, i + 0.5)$ — half a pixel in from the corner.
- 2 Scale into image coordinates on the plane, then **negate** v : rows grow down, `camera.v` points up.
- 3 $\text{Direction} = -\text{camera.d} \cdot \text{camera.w} + u \cdot \text{camera.u} + v \cdot \text{camera.v}$.
- 4 Return `Ray(camera.ey, direction)` — unnormalised, so $t = 1$ lands on the image plane.



$[-0.5 \cdot \text{camera.width}, 0.5 \cdot \text{camera.width}]$
 $\times [-0.5 \cdot \text{camera.height}, 0.5 \cdot \text{camera.height}]$

Origin moves from the top-left corner to the plane's centre; v flips.

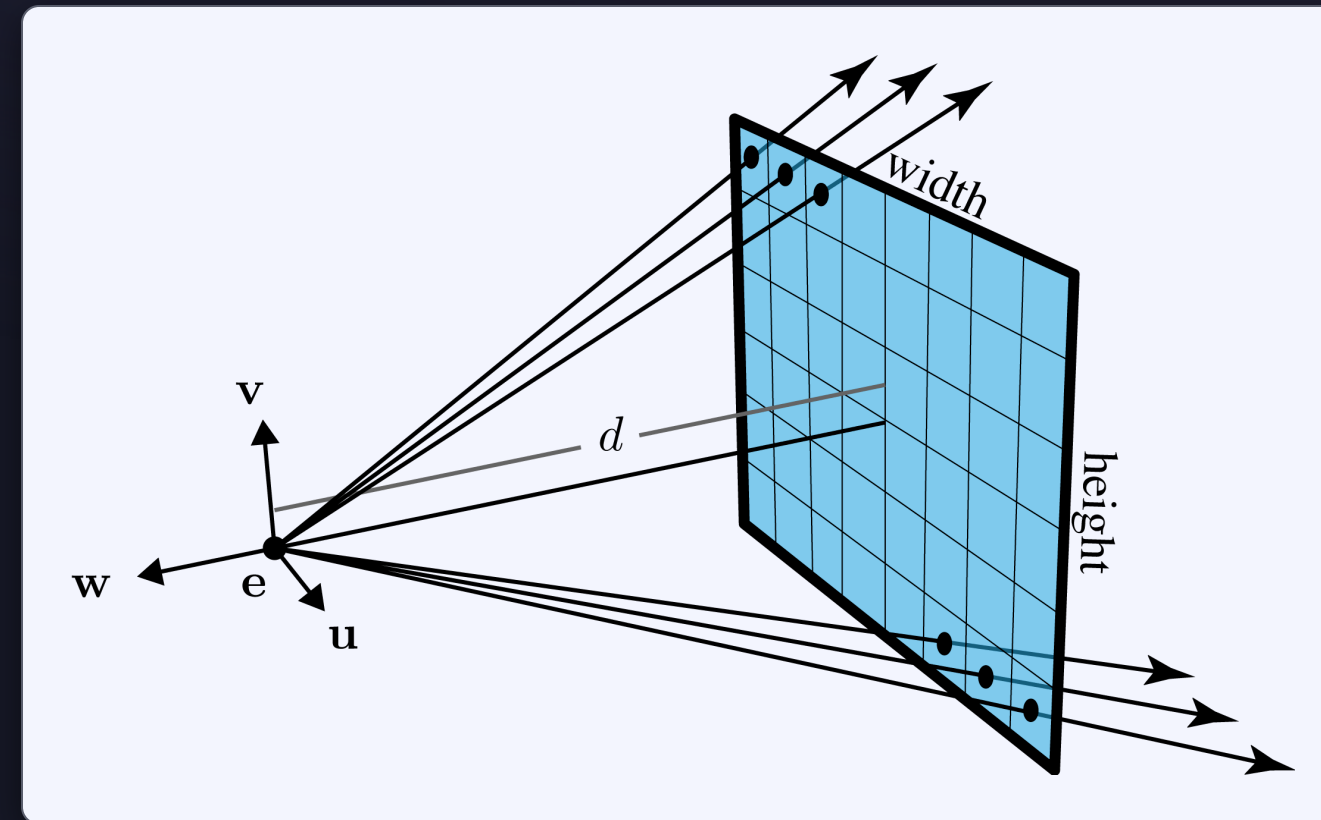
viewing_ray

YOUR TASK — 12 MARKS

- `viewing_ray(camera, i, j, width, height) → a Ray` through pixel (i, j) 's *centre*.

ALGORITHM

- 1 Pixel centre: $(j + 0.5, i + 0.5)$ — half a pixel in from the corner.
- 2 Scale into image coordinates on the plane, then **negate** v : rows grow down, `camera.v` points up.
- 3 $\text{Direction} = -\text{camera.d} \cdot \text{camera.w} + u \cdot \text{camera.u} + v \cdot \text{camera.v}$.
- 4 Return `Ray(camera.eye, direction)` — unnormalised, so $t = 1$ lands on the image plane.



`-camera.d · camera.w` reaches the plane; u and v slide across it.

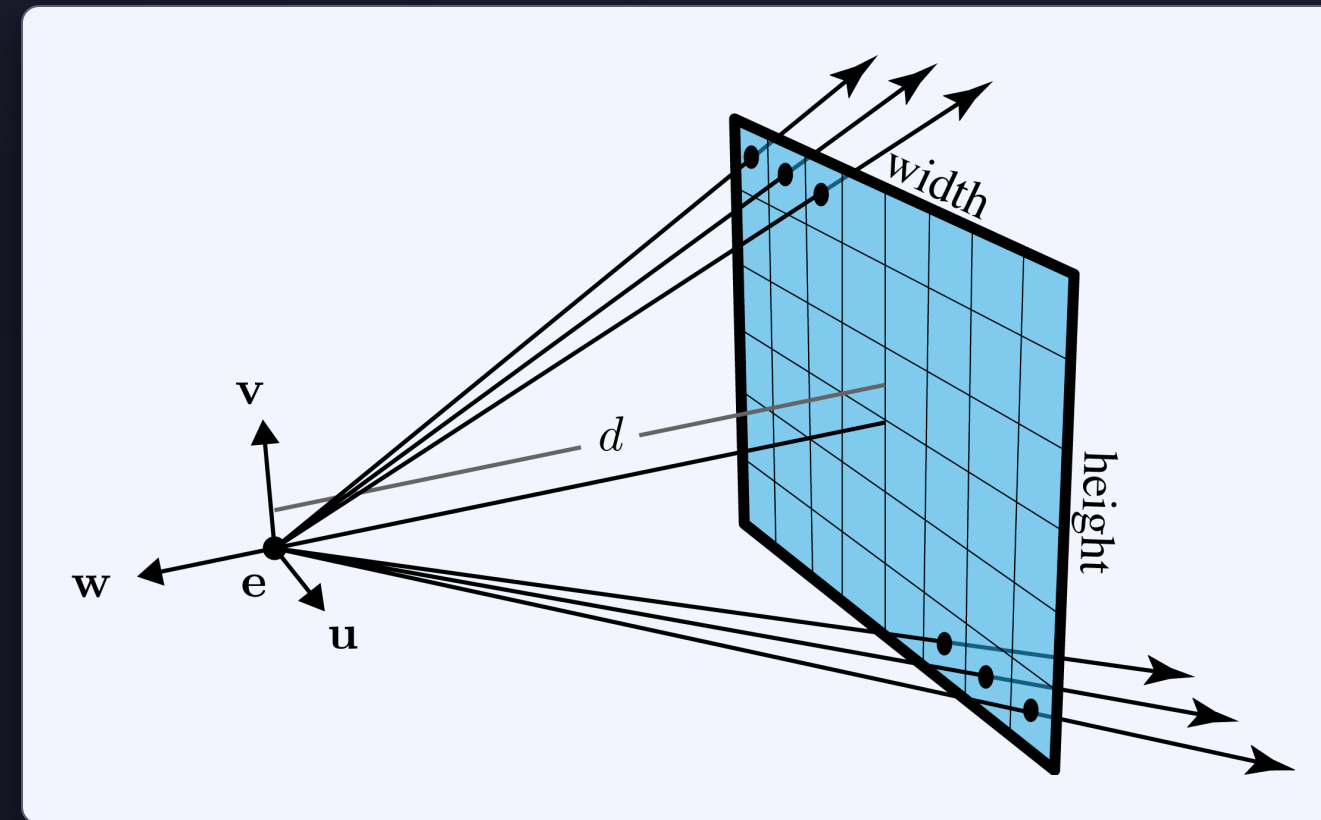
viewing_ray

YOUR TASK — 12 MARKS

- `viewing_ray(camera, i, j, width, height) → a Ray` through pixel (i, j) 's *centre*.

ALGORITHM

- 1 Pixel centre: $(j + 0.5, i + 0.5)$ — half a pixel in from the corner.
- 2 Scale into image coordinates on the plane, then **negate** v : rows grow down, `camera.v` points up.
- 3 $\text{Direction} = -\text{camera.d} \cdot \text{camera.w} + u \cdot \text{camera.u} + v \cdot \text{camera.v}$.
- 4 Return `Ray(camera.eye, direction)` — **un-normalised**, so $t = 1$ lands on the image plane.



Every ray leaves `camera.eye` through its pixel's centre; $t = 1$ is the image plane.

```
# running example · sphere-and-plane.json
eye=(0,0,5) d=3 camera.width=1.78 camera.height=1
axis ray: origin (0,0,5) direction (0,0,-3)
```

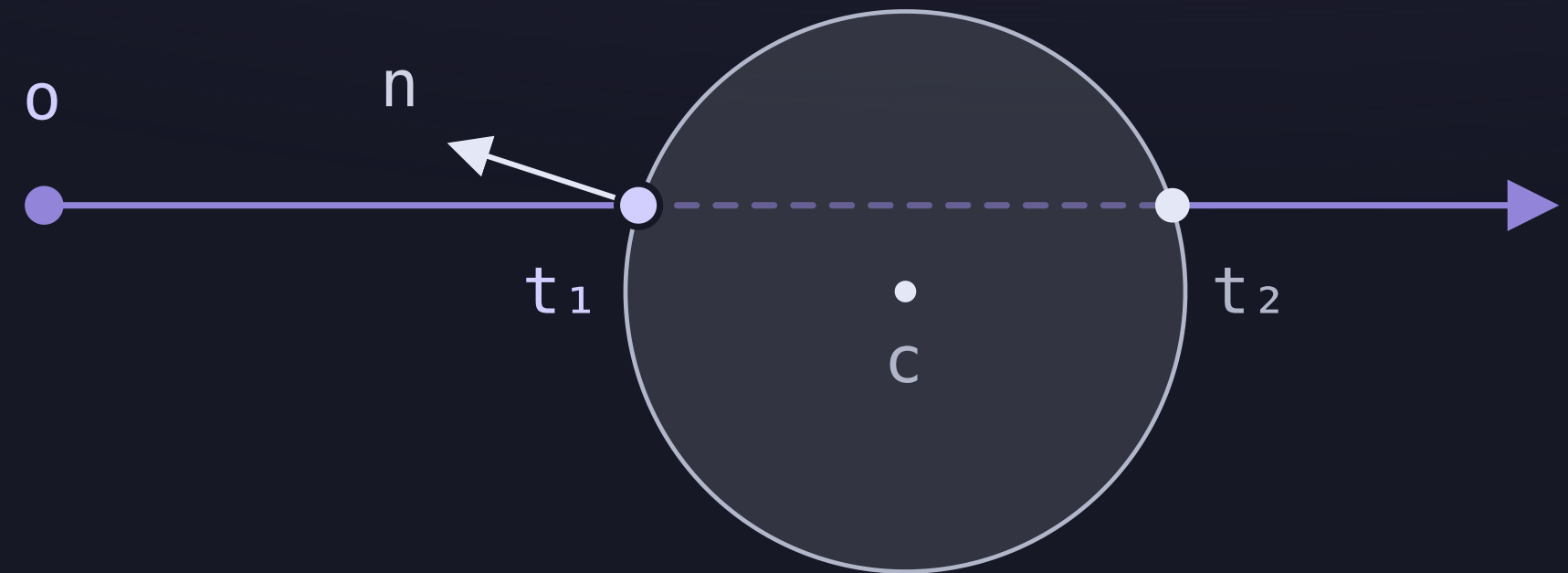
sphere_intersect

CONCEPT

- On the sphere when $|q - c|^2 = r^2$. Substitute $q = o + t \cdot d$ for a quadratic in t .

YOUR TASK — 12 MARKS

- `sphere_intersect(center, radius, ray, min_t) → (hit, t, n)`. Near root first; if it is $< \text{min_t}$ you are *inside* — use the far root.



Keep t_1 ; if $t_1 < \text{min_t}$ the origin is *inside* — keep t_2 .

```
# c=(0,0,0) r=0.5
A=9 B=-30 C=24.75
disc=9 → t=1.5 ✓
hit (0,0,0.5)
n=(0,0,1)
```



inside-a-sphere — the far root

plane_intersect

CONCEPT

- A plane is a point p_0 and a normal n : every q with $(q - p_0) \cdot n = 0$. Linear in t — at most one hit.

YOUR TASK — 8 MARKS

- `plane_intersect(point, normal, ray, min_t)`
→ `(hit, t, n)`. Parallel ($d \cdot n = 0$) is no hit; reject $t < \text{min_t}$.



At most one hit: $t = -((o - p_0) \cdot n) / (d \cdot n)$. $d \cdot n = 0 \rightarrow$ parallel, no hit.

```
# floor: p_0=(0,-0.5,0)  n=(0,1,0)
axis ray d=(0,0,-3):    d @ n = 0  → miss
tilt d=(0,-0.2,-3):    t = -0.5 / -0.2 = 2.5
```

triangle_intersect

CONCEPT

- A point on the triangle is $a + \beta(b-a) + \gamma(c-a)$.
Equate with $o + t \cdot d$: three unknowns (β, γ, t), one 3×3 system.

YOUR TASK — 12 MARKS

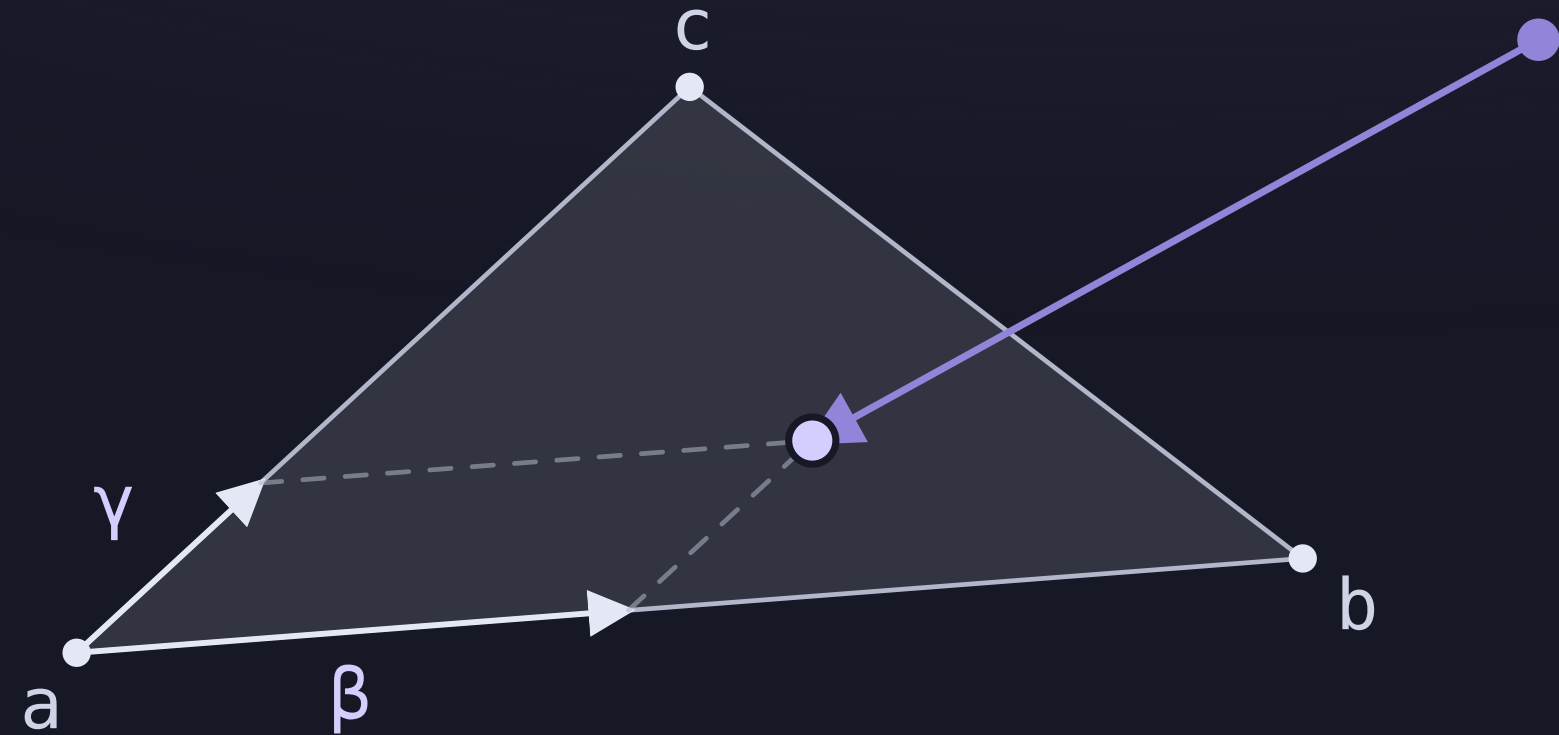
- `triangle_intersect(a, b, c, ray, min_t) → (hit, t, n)` by Cramer's rule. Normal: `normalize((a - b) × (a - c))` — this order.

CRAMER'S RULE

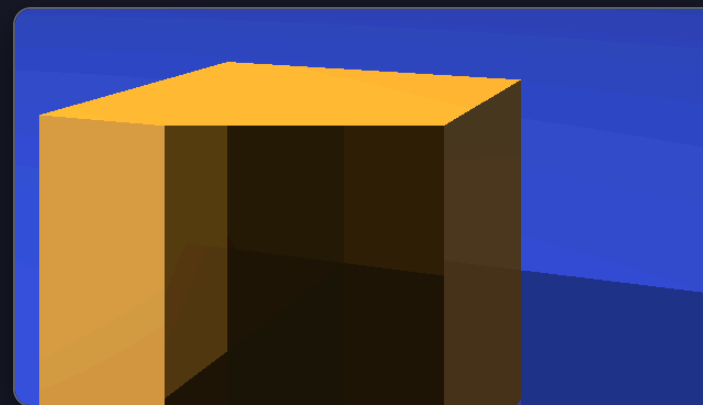
$$Ax = b$$

$$x_i = \frac{\det A_i}{\det A}$$

- A_i is A with its i -th column replaced by b .



Inside when $\beta, \gamma \geq 0$ and $\beta + \gamma \leq 1$ — and $t \geq \text{min_t}$.



cube-and-plane — twelve triangles
from `cube.stl`

`_intersect_object` and triangle soups

CONCEPT

- One object, many triangles: its hit is the **nearest among them** — the same `min_t` rule, one level down.

YOUR TASK — 10 MARKS, WITH `FIRST_HIT`

- `_intersect_object(obj, ray, min_t)`: dispatch on `obj.geometry.kind`; "soup" loops its (n, 3, 3) triangles.

ALGORITHM

- 1 Dispatch on `obj.geometry.kind` to the routine you already wrote.
- 2 "soup": loop its (n, 3, 3) triangles — smallest $t \geq min_t$ wins.



`bunny.json` — a 1000-triangle soup

"accelerate": true pre-filters triangles a ray cannot hit — your code and the image do not change.

first_hit

CONCEPT

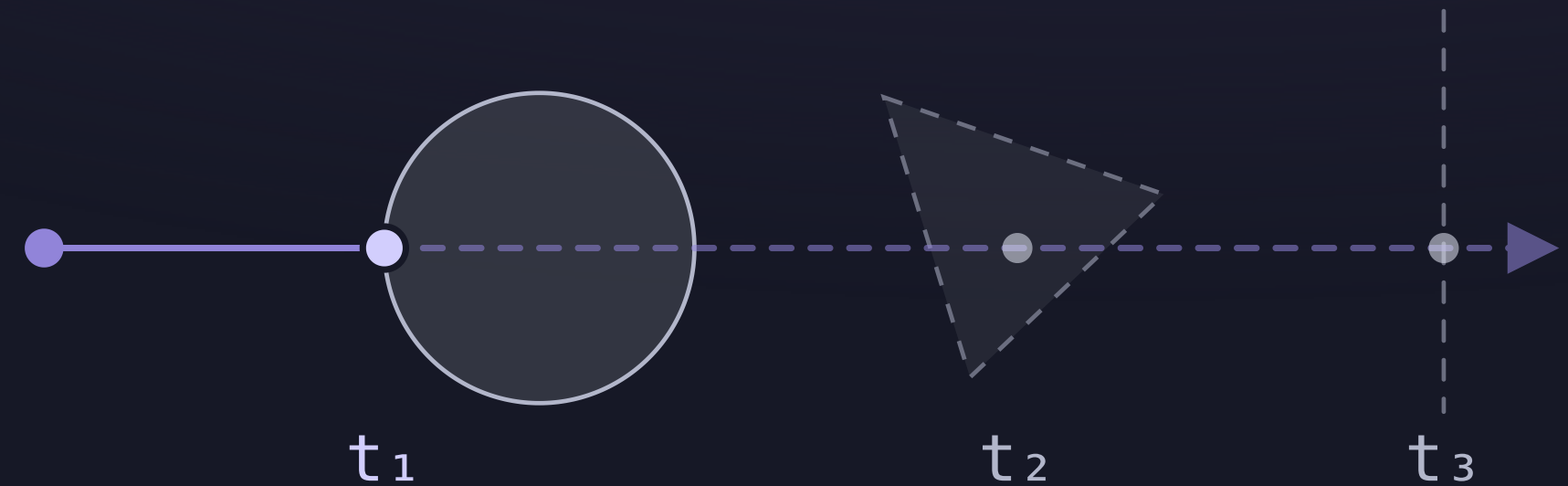
- Visibility is a **minimum**: the smallest $t \geq \text{min_t}$ wins, everything behind it is hidden.

YOUR TASK — 10 MARKS

- `first_hit(ray, min_t, objects) → (hit, hit_id, t, n)`: the smallest t over every object.

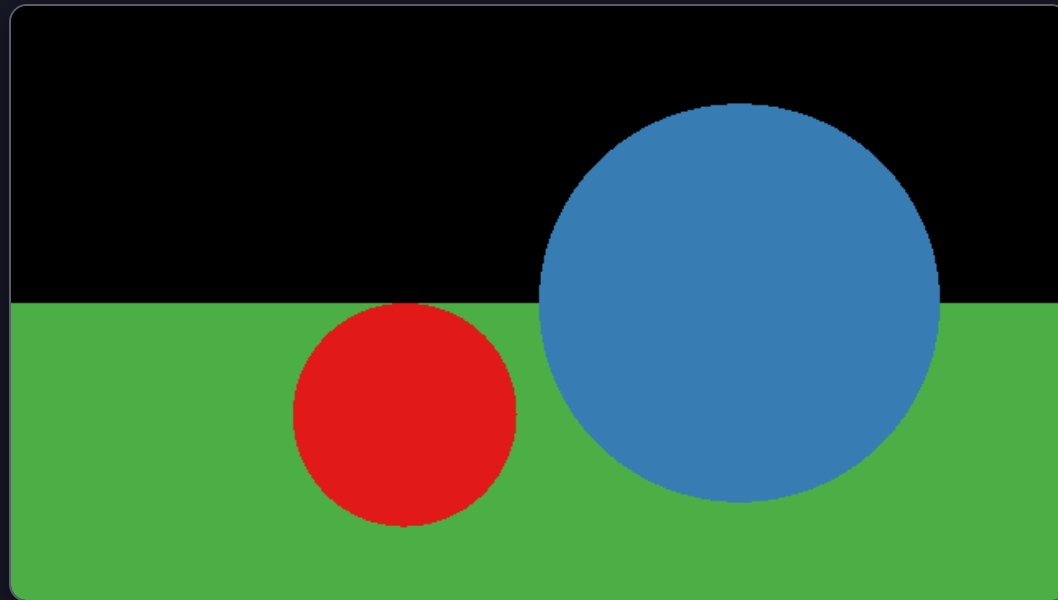
ALGORITHM

- 1 Start from "nothing hit": $\text{hit_id} = -1, t = \text{inf}$.
- 2 Ask each object for its own nearest hit via `_intersect_object`.
- 3 Keep the hit with the smallest $t \geq \text{min_t}$, which object it belonged to, and the normal there.
- 4 Nothing struck $\rightarrow (\text{False}, -1, \text{inf}, \text{zeros}(3))$.

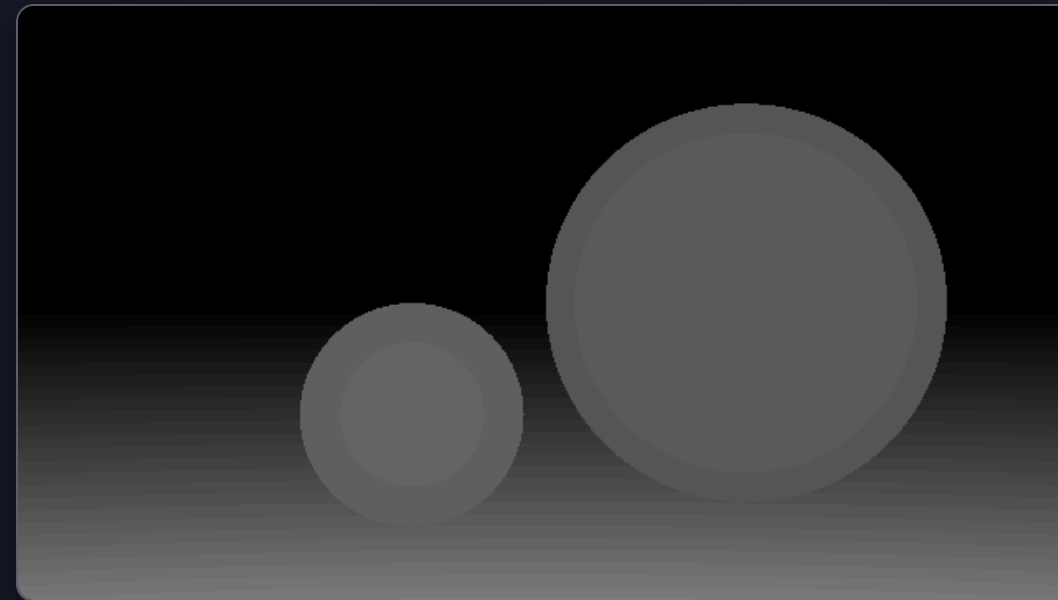


Three candidates, one ray — only the smallest $t \geq \text{min_t}$ is seen.

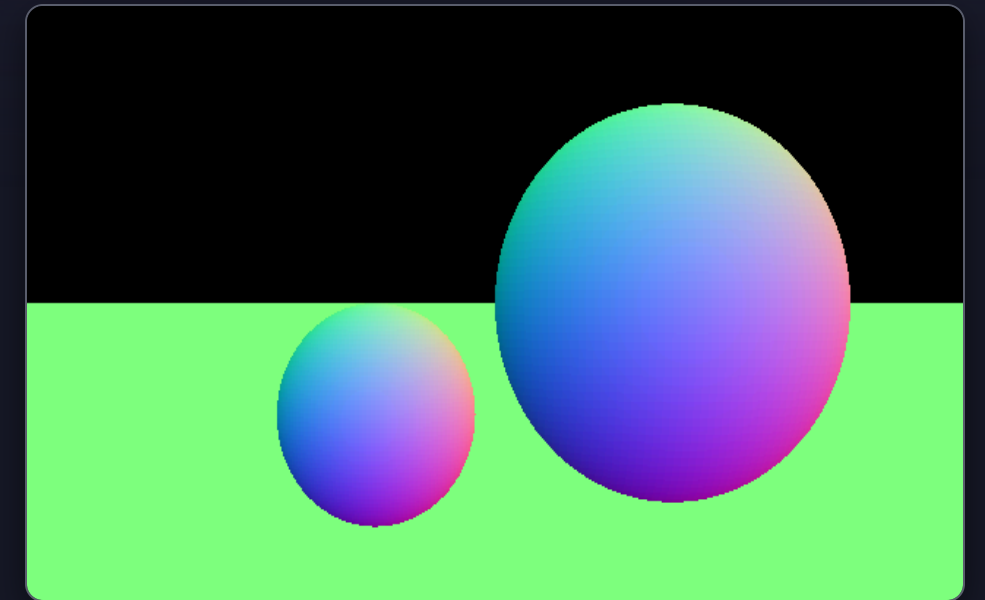
Reading the Part 1 buffers



object id — one flat colour each.
Wrong silhouettes → `viewing_ray` or
an intersection.



depth — `t` as grey, nearer is brighter.
Flat, or fading the wrong way → the wrong root,
or `min_t`.

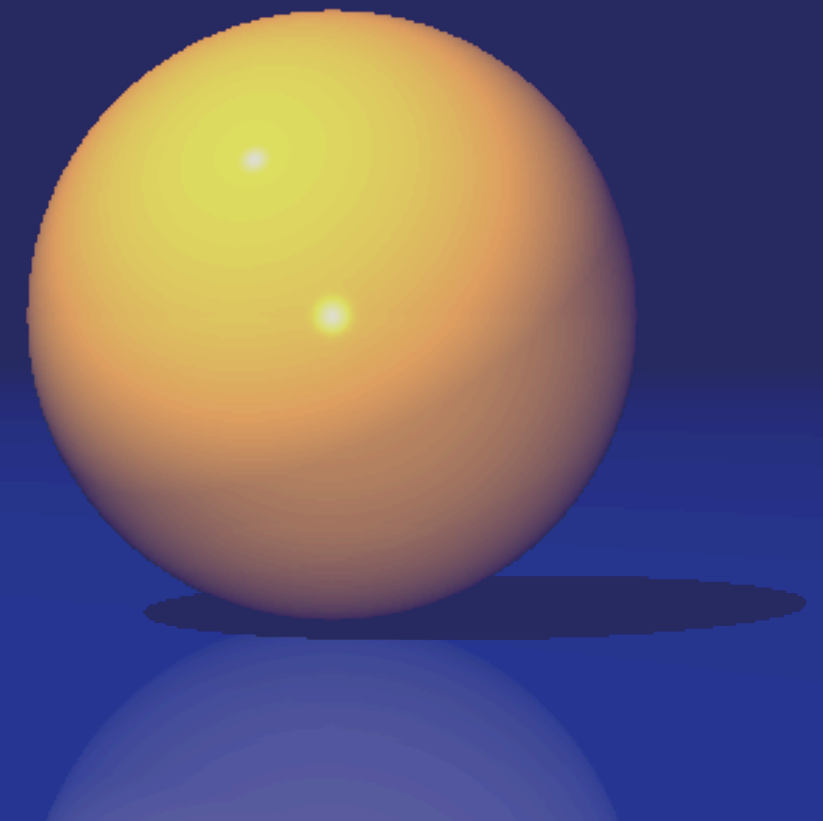


normals — $\text{rgb} = \frac{1}{2} \cdot \mathbf{n} + \frac{1}{2}$.
Bands or noise → `n` not unit, or flipped.

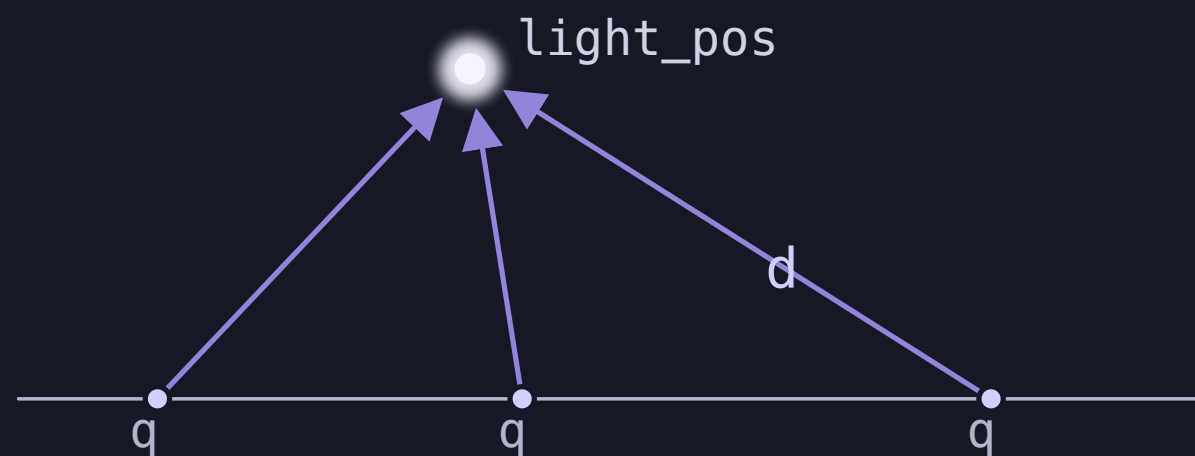
```
$ python main.py --part 1      # run after every intersection routine – no lighting code needed
```

04

— Part 2: Ray tracing



Two light helpers — direction toward the light, and how far



$d = \text{light_pos} - q$
each d points at the light · $|d|$ varies

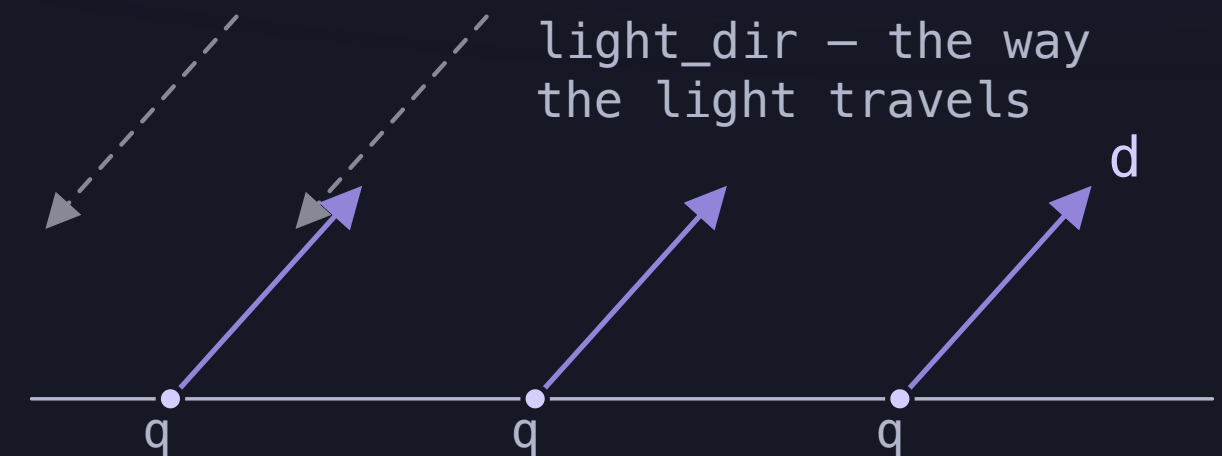
point_light_direction

CONCEPT

- The light has a position, so the direction depends on *where you are*: the vector from q to it.

YOUR TASK — 6 MARKS

- $(\text{light_pos} - q, 1.0)$ — *not* normalised, so $\text{max_t} = 1$ is exactly the light.



$d = -\text{normalize}(\text{light_dir})$
 d points back along light_dir · $\text{max_t} = \infty$

directional_light_direction

CONCEPT

- Infinitely far away: every point sees the *same* direction — so $\text{max_t} = \infty$.

YOUR TASK — 6 MARKS

- light_dir is the way the light *travels*; toward the light is its negative. q is unused.

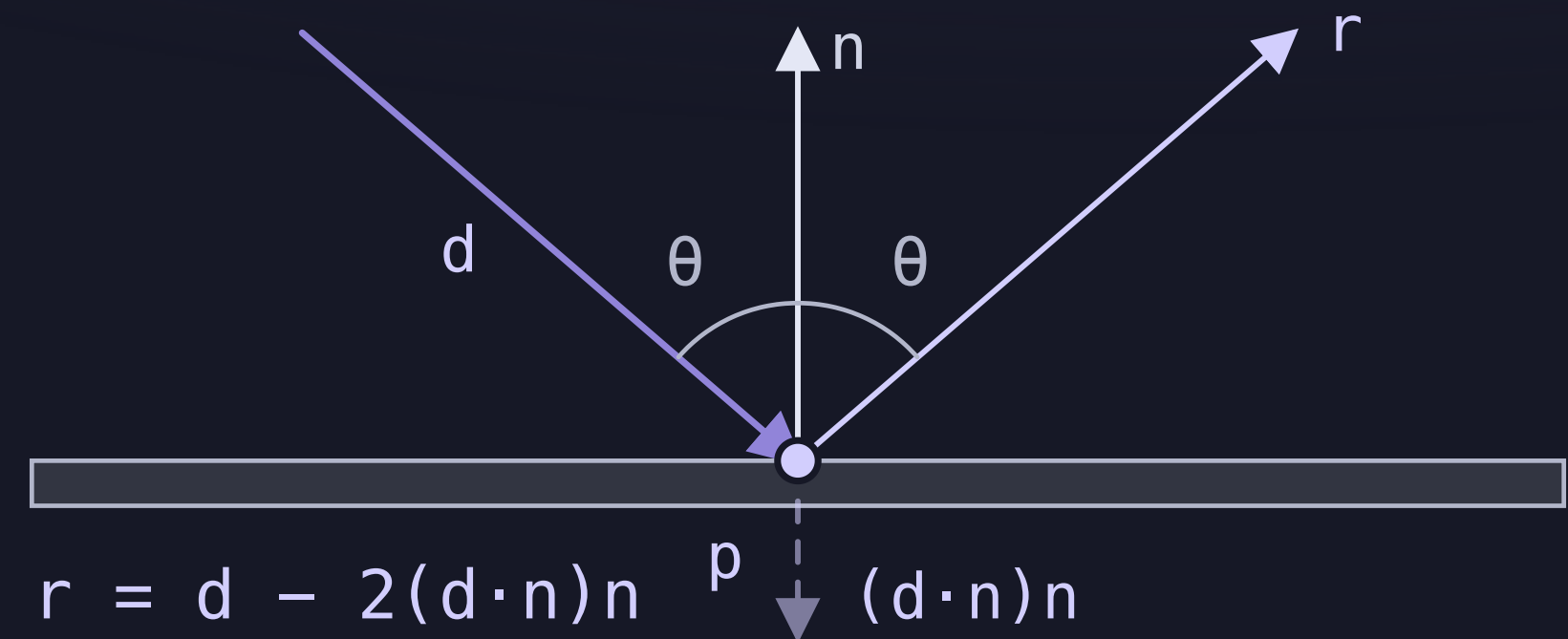
reflect

CONCEPT

- $(d \cdot n)n$ is the part of d along n . Subtract it **twice**: that part flips, the rest is untouched.

YOUR TASK — 6 MARKS

- $\text{reflect}(d, n) \rightarrow d - 2 (d \cdot n) n$. n . n unit; the result keeps $|d|$ — do *not* normalise.

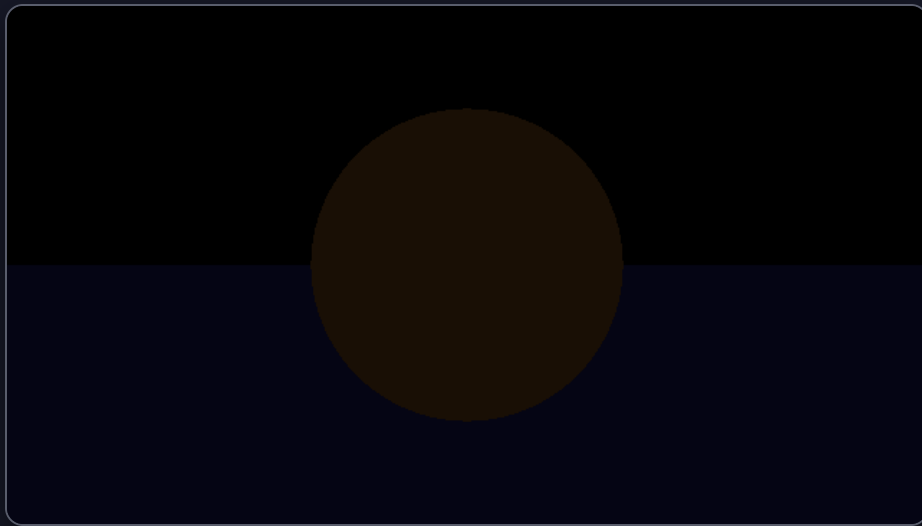


d points into the surface, so $d \cdot n < 0$ and $(d \cdot n)n$ points inward — subtracting it twice sends the ray back out at the same angle, with $|r| = |d|$.

```
# running example · the sphere's front point  
d=(0,0,-3) n=(0,0,1) d@n=-3 → r=(0,0,3)
```

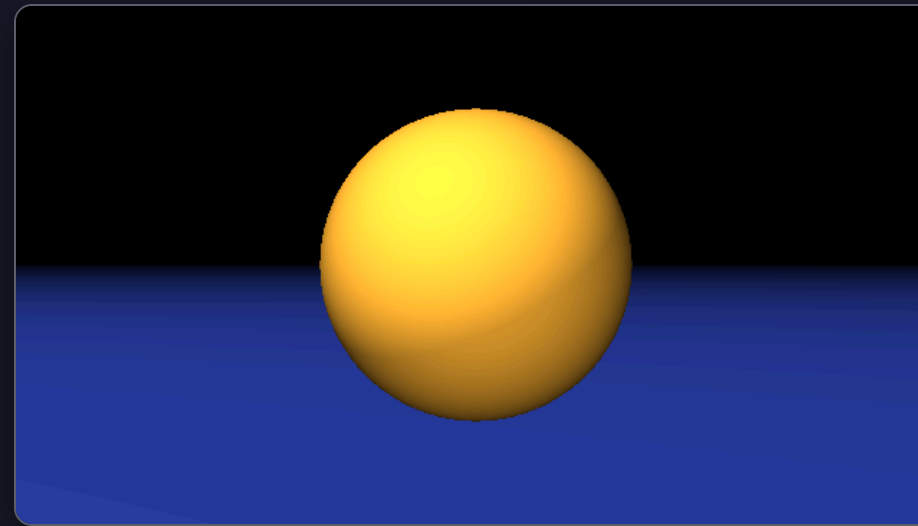
blinn_phong_shading — three terms

$$L += 0.1 \cdot k_a$$



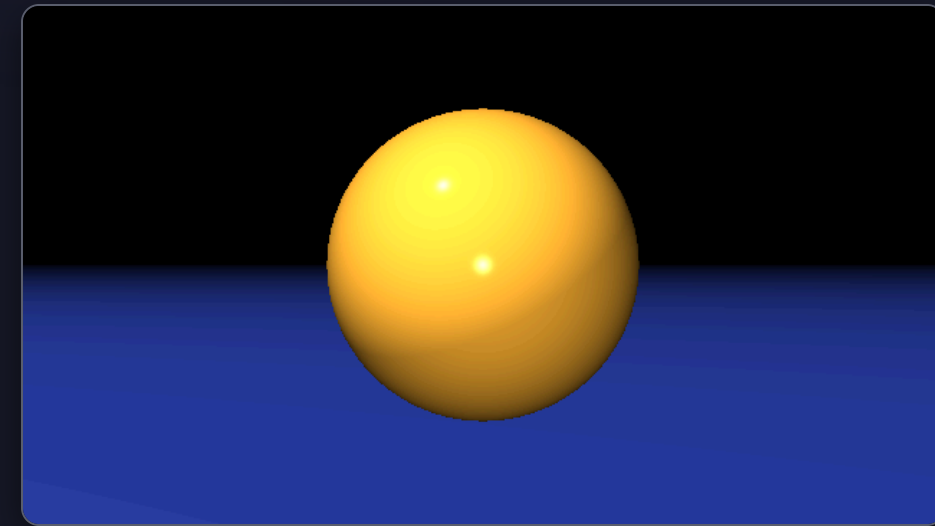
ambient — once, not per light

$$L += I \cdot k_d \cdot \max(0, n \cdot l)$$



+ **diffuse** — per lit light, zero past 90°

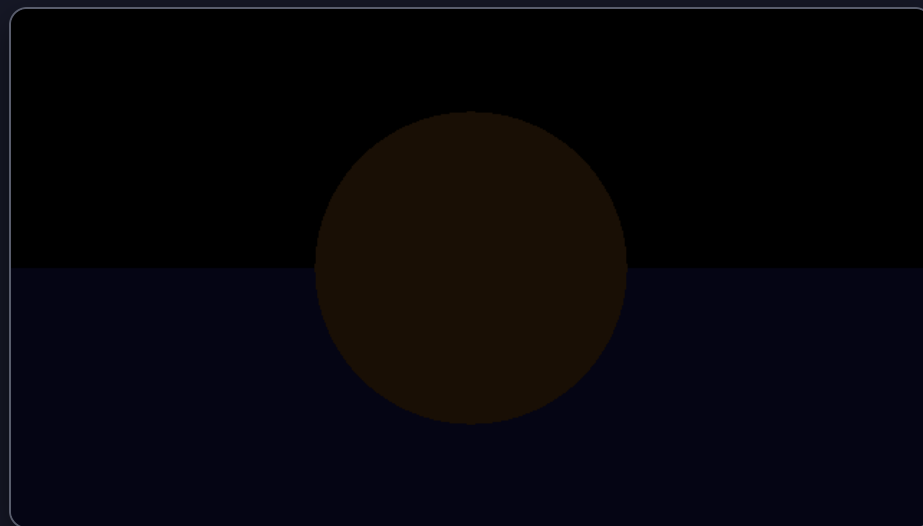
$$L += I \cdot k_s \cdot \max(0, n \cdot h)^p$$



+ **specular** — p sets how tight the highlight is

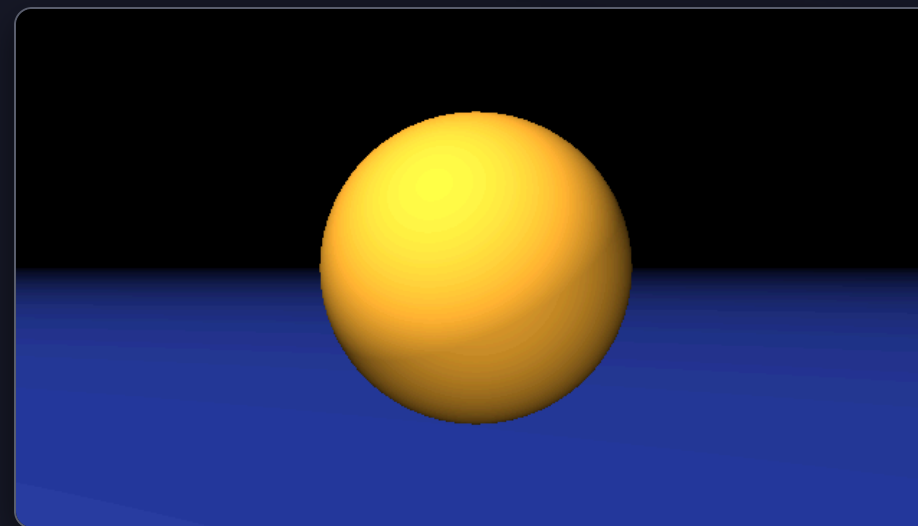
blinn_phong_shading — the two factors, up close

$L += 0.1 \cdot k_a$



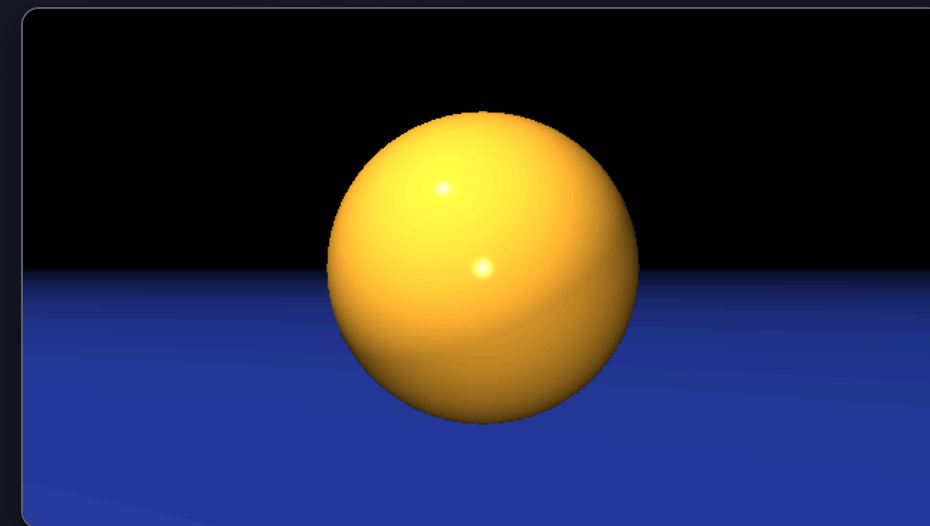
ambient — once, not per light

$L += I \cdot k_d \cdot \max(0, n \cdot l)$

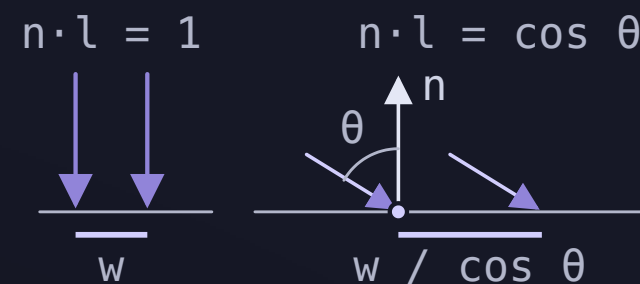


+ diffuse — per lit light, zero past 90°

$L += I \cdot k_s \cdot \max(0, n \cdot h)^p$



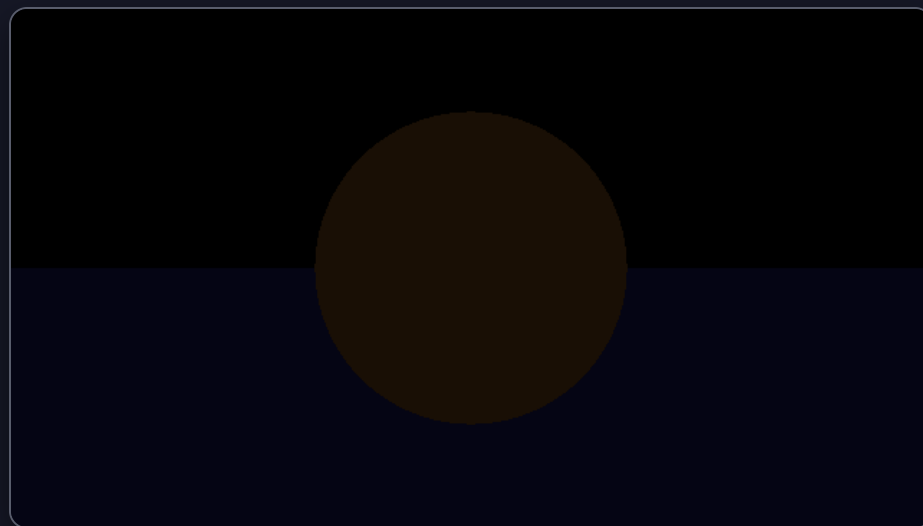
+ specular — p sets how tight the highlight is



cosine foreshortening · $n \cdot l$
 $= \cos \theta$

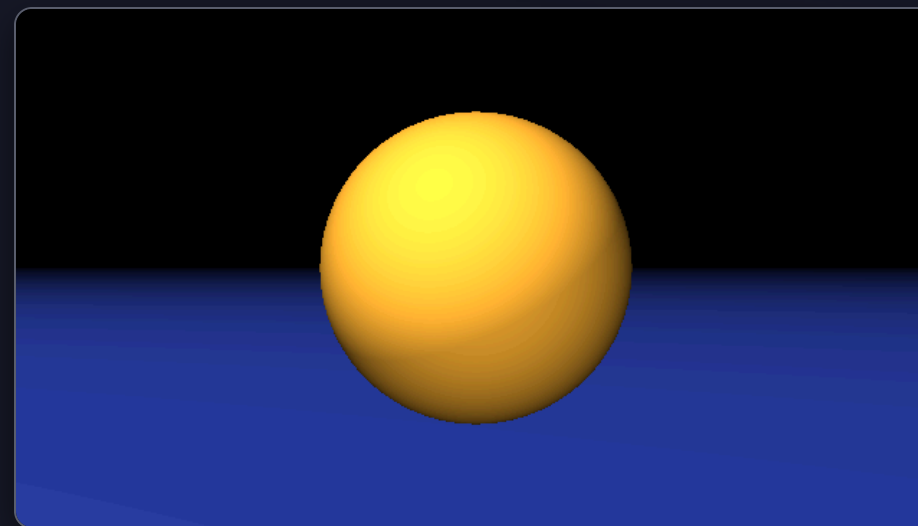
blinn_phong_shading — the two factors, up close

$$L += 0.1 \cdot ka$$



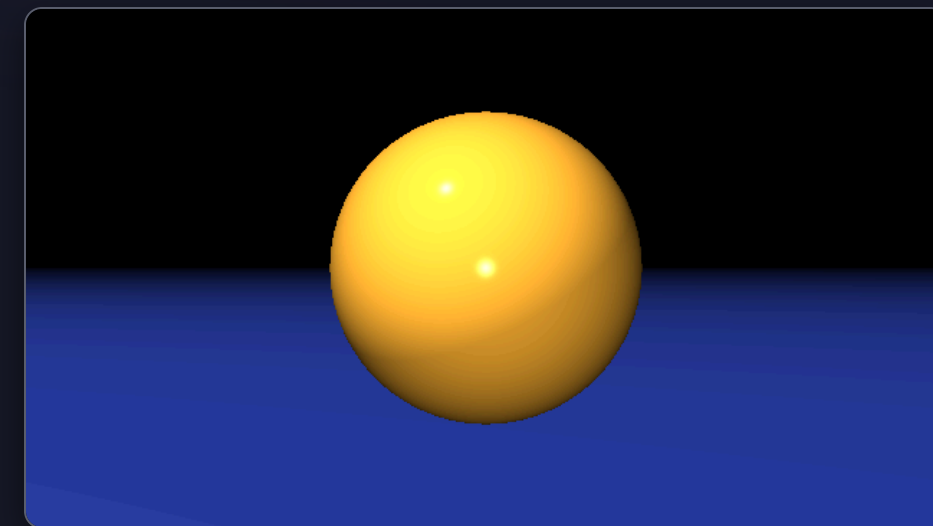
ambient — once, not per light

$$L += I \cdot kd \cdot \max(0, n \cdot l)$$

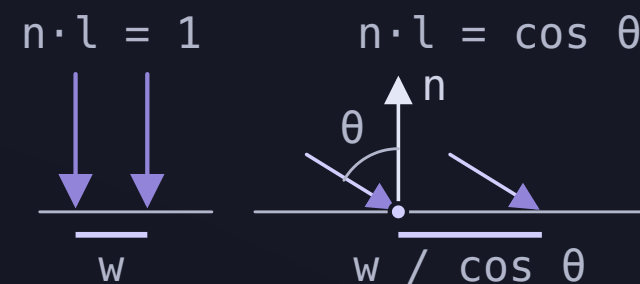


+ diffuse — per lit light, zero past 90°

$$L += I \cdot ks \cdot \max(0, n \cdot h)^p$$



+ specular — p sets how tight the highlight is



cosine foreshortening · $n \cdot l = \cos \theta$



$p = 20$



$p = 1000$

phong_exponent · bigger p , tighter dot

blinn_phong_shading — the example ray, shaded

CONCEPT

AMBIENT

```
L += 0.1 · ka
ka = (1, 0.7, 0.2)
= (0.10, 0.07, 0.02)
```

+

DIRECTIONAL LIGHT · I

```
= 0.8
L += I · kd · n · l +
I · ks · n · h^p
n · l = 1 · n · h = 1
= (0.80, 0.56, 0.16)
+ 0.64
```

+

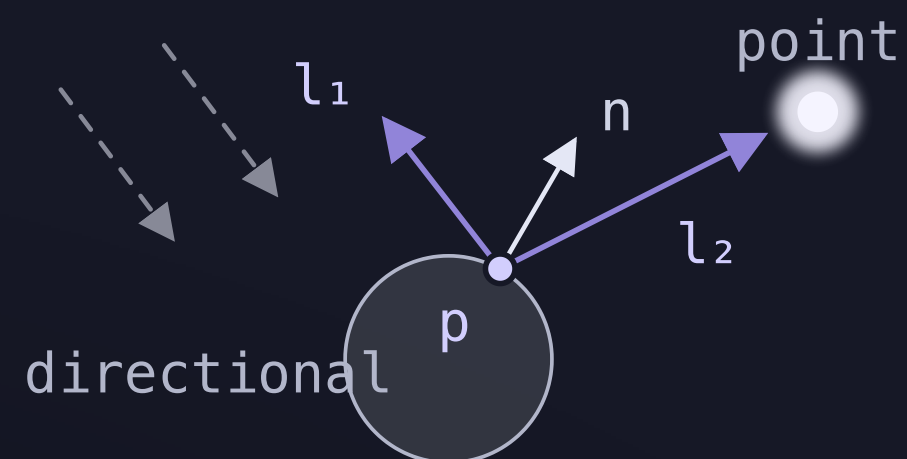
POINT LIGHT · I = 0.8

```
L += I · kd · n · l +
I · ks · n · h^p
n · l ≈ 0.39 · n · h
≈ 0.83
= (0.31, 0.22, 0.06)
+ 0
```

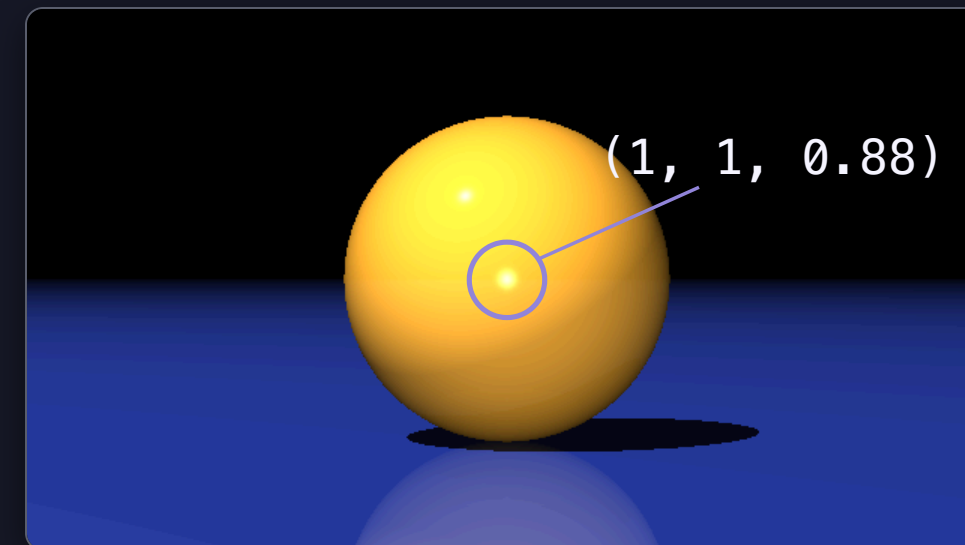
=

PIXEL

```
L = (1.85,
1.49, 0.88)
→ clamp → (1, 1, 0.88)
```



both lights contribute its own $n \cdot l$ and $n \cdot h$



sphere-and-plane — this pixel

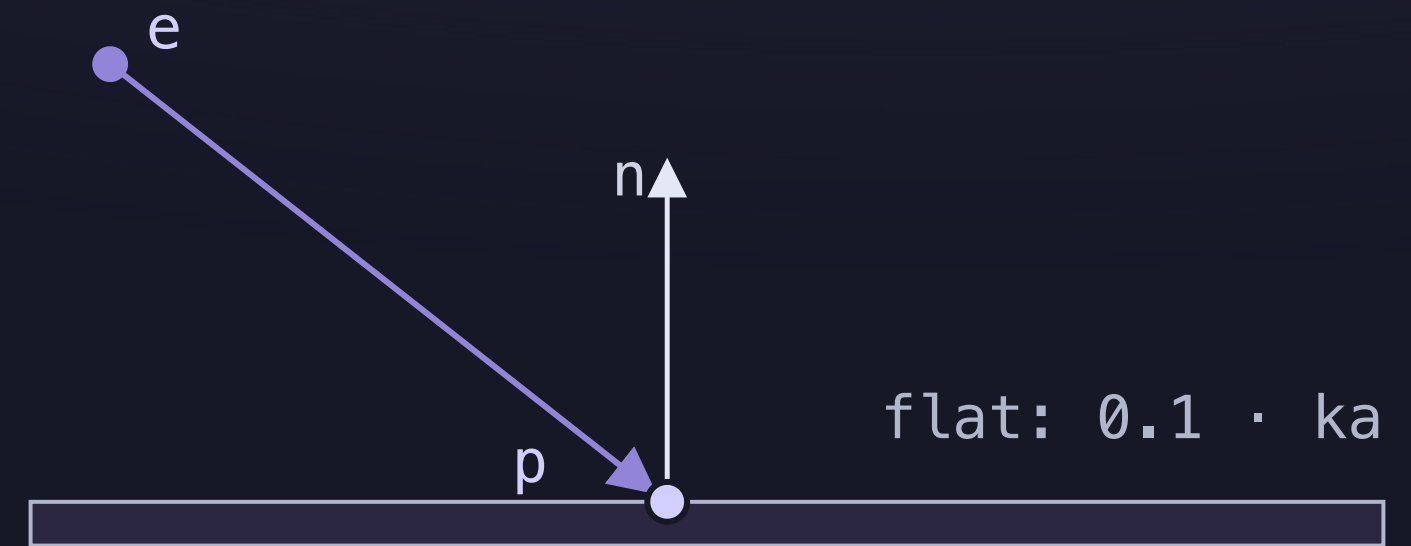
blinn_phong_shading

YOUR TASK — 14 MARKS

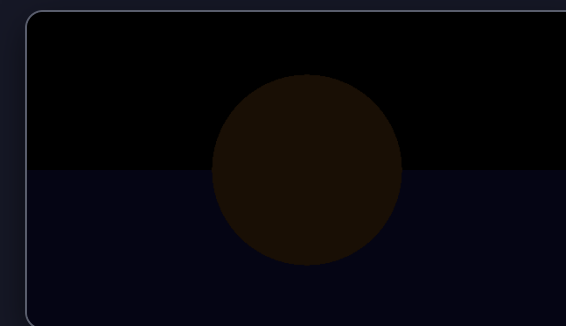
- `blinn_phong_shading(ray, hit_id, t, n, objects, lights)`
→ `rgb`: the surface colour, summed over every light that can see the hit point.

ALGORITHM

- 1 Hit point $p = \text{ray.origin} + t \cdot \text{ray.direction}$.
- 2 Ambient, added **once**: $0.1 \cdot m.k_a$ — m is the hit material.
- 3 For each light, `light_direction(light, p) → (d, max_t)`: toward the light, and how far.
- 4 Shadow ray from p along d , offset $1e-6$. Anything before max_t → blocked, skip that light.
- 5 Otherwise add diffuse $m.k_d \cdot I \cdot \max(0, n \cdot l)$ and specular $m.k_s \cdot I \cdot \max(0, n \cdot h)^p$.
- 6 Return the sum un-clamped.



$$p = \text{ray.origin} + t \cdot \text{ray.direction}$$



ambient only — flat, no direction

$m =$
`objects[hit_id].material:`
`ka, kd, ks, km` and
`phong_exponent`. Colours
multiply component-wise.

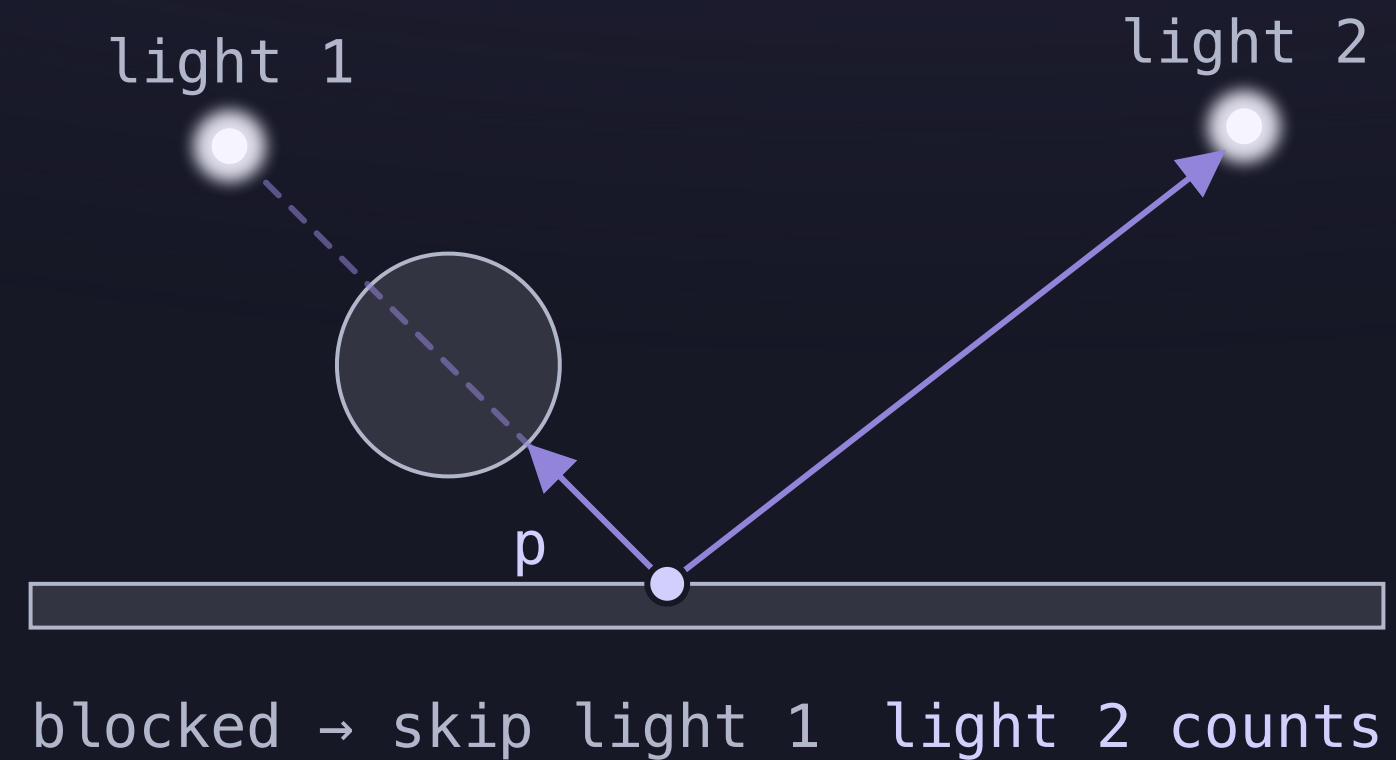
blinn_phong_shading

YOUR TASK — 14 MARKS

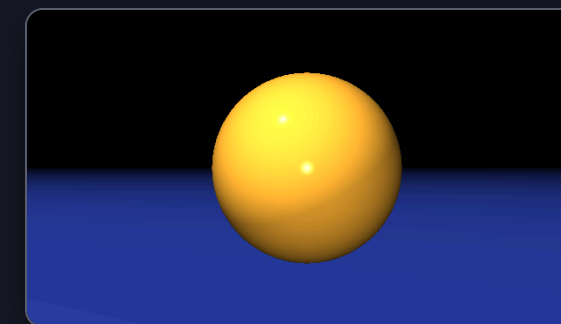
- `blinn_phong_shading(ray, hit_id, t, n, objects, lights)`
→ `rgb`: the surface colour, summed over every light that can see the hit point.

ALGORITHM

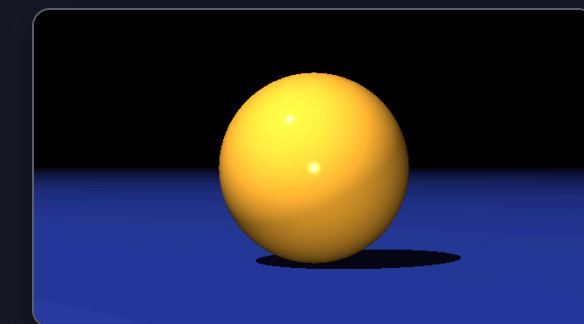
- 1 Hit point $p = \text{ray.origin} + t \cdot \text{ray.direction}$.
- 2 Ambient, added once: $0.1 \cdot m.k_a$ — m is the hit material.
- 3 For each light, `light_direction(light, p)` → (d , max_t): toward the light, and how far.
- 4 Shadow ray from p along d , offset $1e-6$. Anything before max_t → blocked, skip that light.
- 5 Otherwise add diffuse $m.k_d \cdot I \cdot \max(0, n \cdot l)$ and specular $m.k_s \cdot I \cdot \max(0, n \cdot h)^p$.
- 6 Return the sum un-clamped.



The epsilon: a shadow ray starting exactly at p can re-hit its own surface at $t \approx 0$ — black speckles. Start $1e-6$ along d .



before — no shadow test



after — with shadows

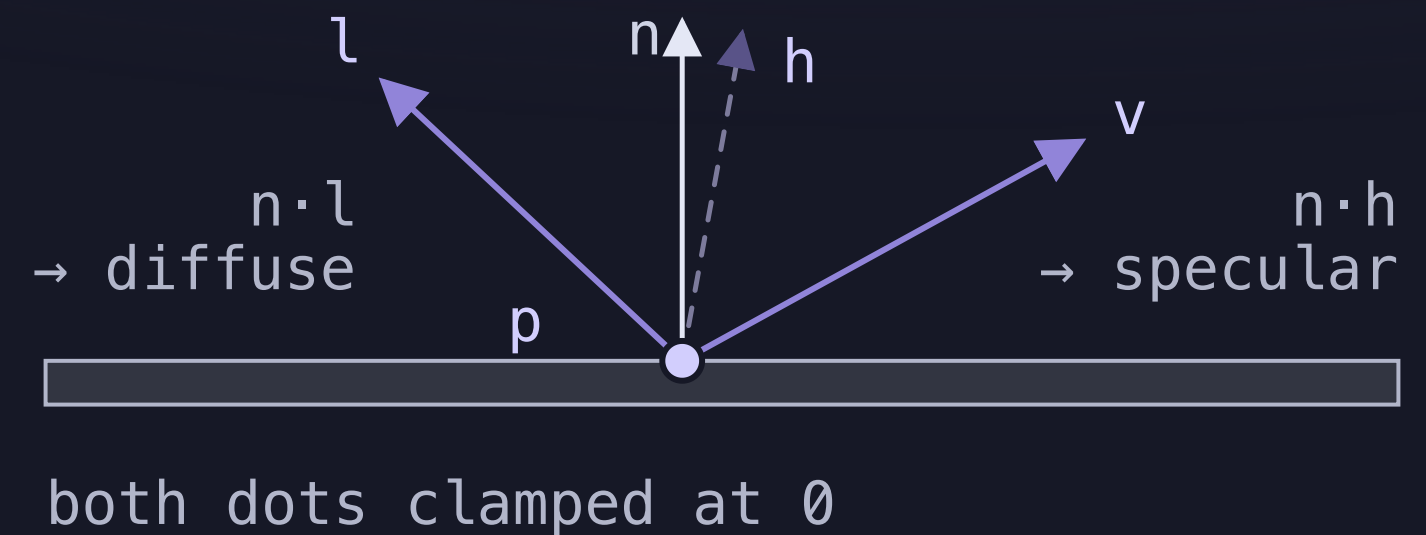
blinn_phong_shading

YOUR TASK — 14 MARKS

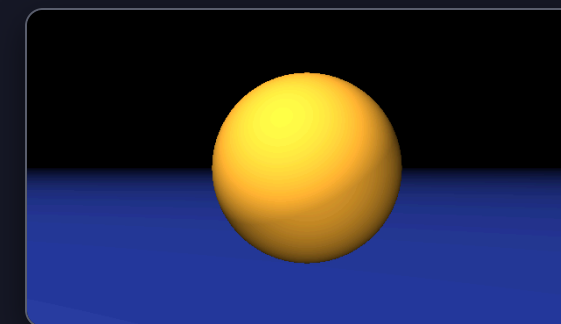
- `blinn_phong_shading(ray, hit_id, t, n, objects, lights)`
→ `rgb`: the surface colour, summed over every light that can see the hit point.

ALGORITHM

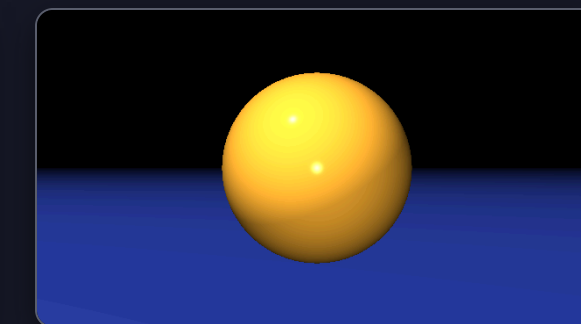
- 1 Hit point $p = \text{ray.origin} + t \cdot \text{ray.direction}$.
- 2 Ambient, added **once**: $0.1 \cdot m.k_a$ — m is the hit material.
- 3 For each light, `light_direction(light, p) → (d, max_t)`: toward the light, and how far.
- 4 Shadow ray from p along d , offset $1e-6$. Anything before max_t → blocked, skip that light.
- 5 Otherwise add diffuse $m.k_d \cdot I \cdot \max(0, n \cdot l)$ and specular $m.k_s \cdot I \cdot \max(0, n \cdot h)^p$.
- 6 Return the sum **un-clamped**.



$l = \text{normalize}(d); v = -\text{normalize}(\text{ray.direction})$ — toward the eye; $h = \text{normalize}(l + v)$. Clamp both dots with $\max(0, \cdot)$.



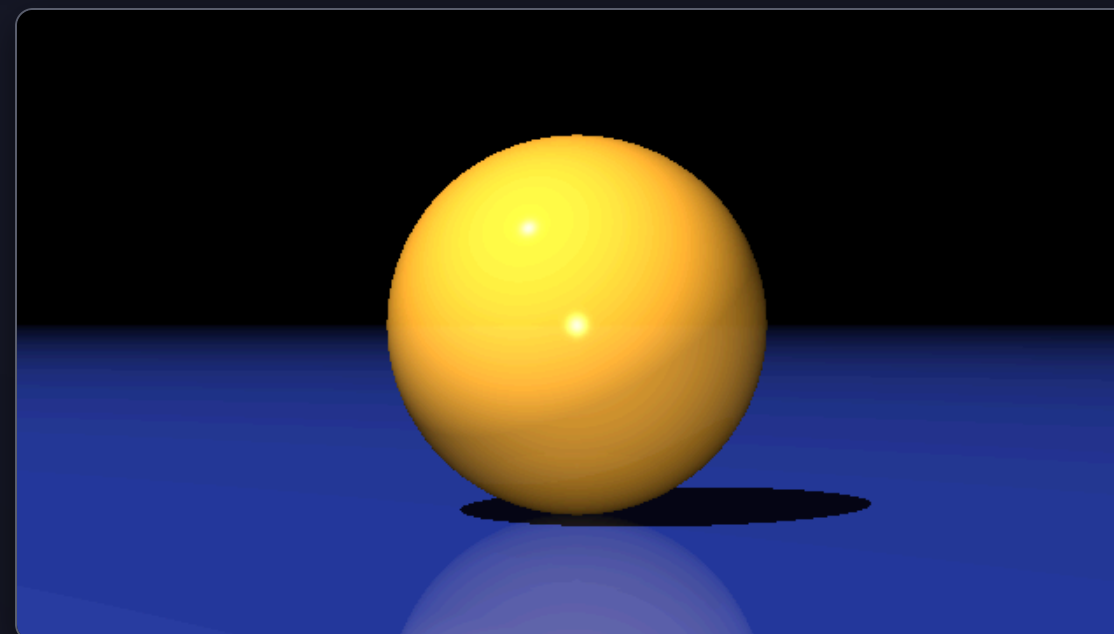
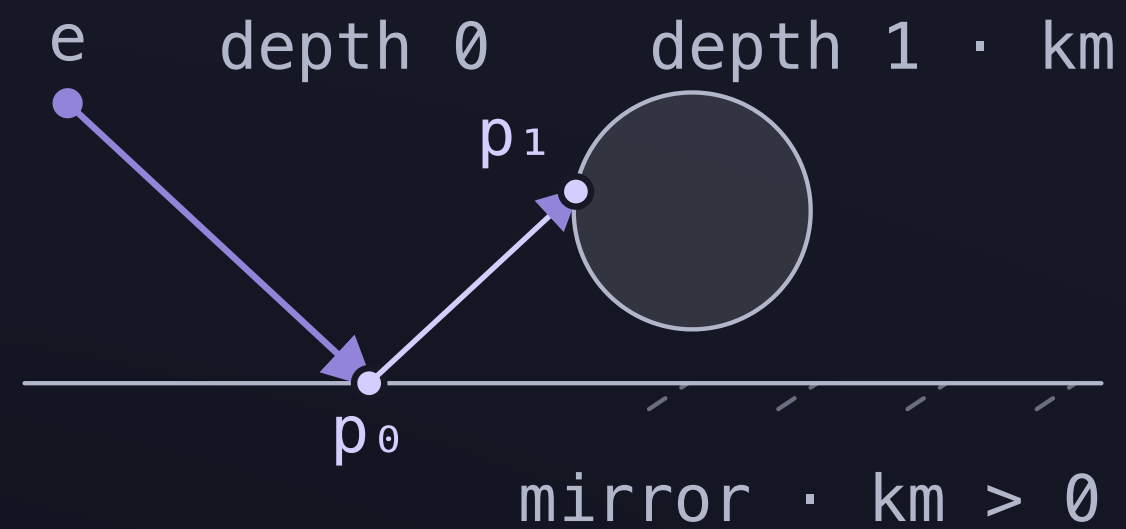
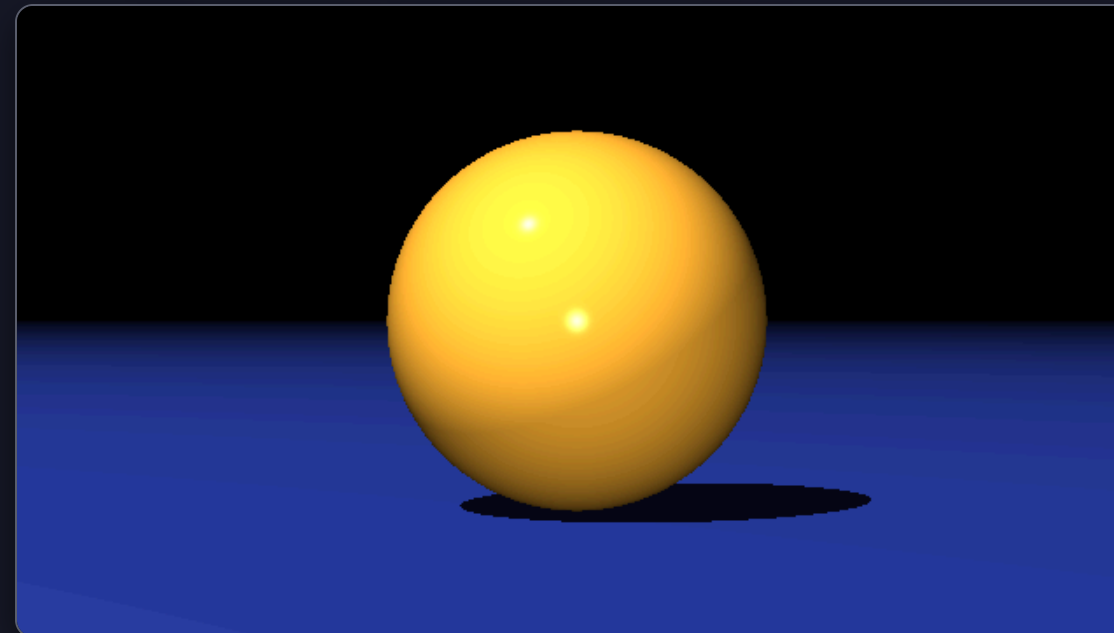
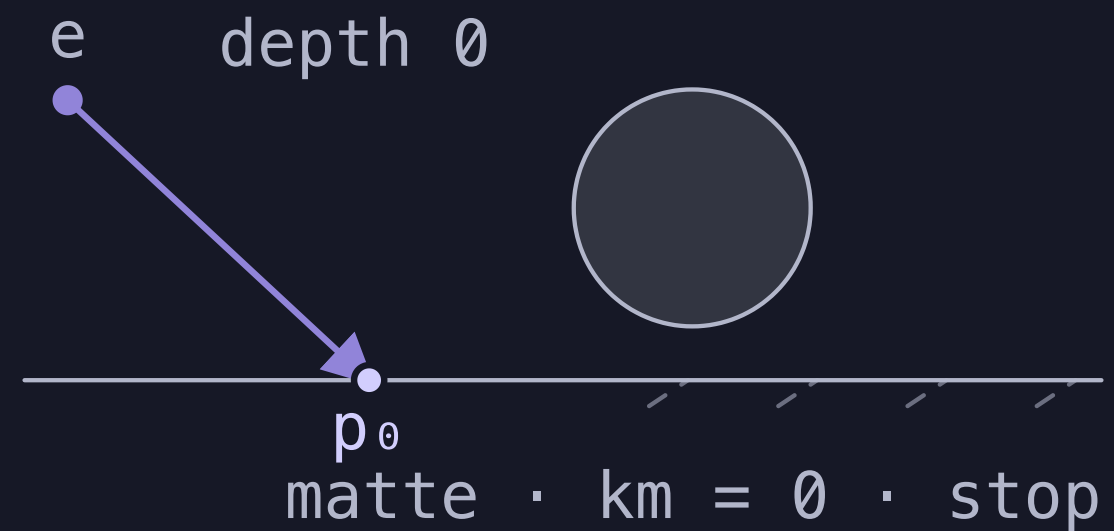
+ diffuse $\cdot n \cdot l$



+ specular $\cdot (n \cdot h)^p$

raycolor — shading, plus one bounce deeper

CONCEPT



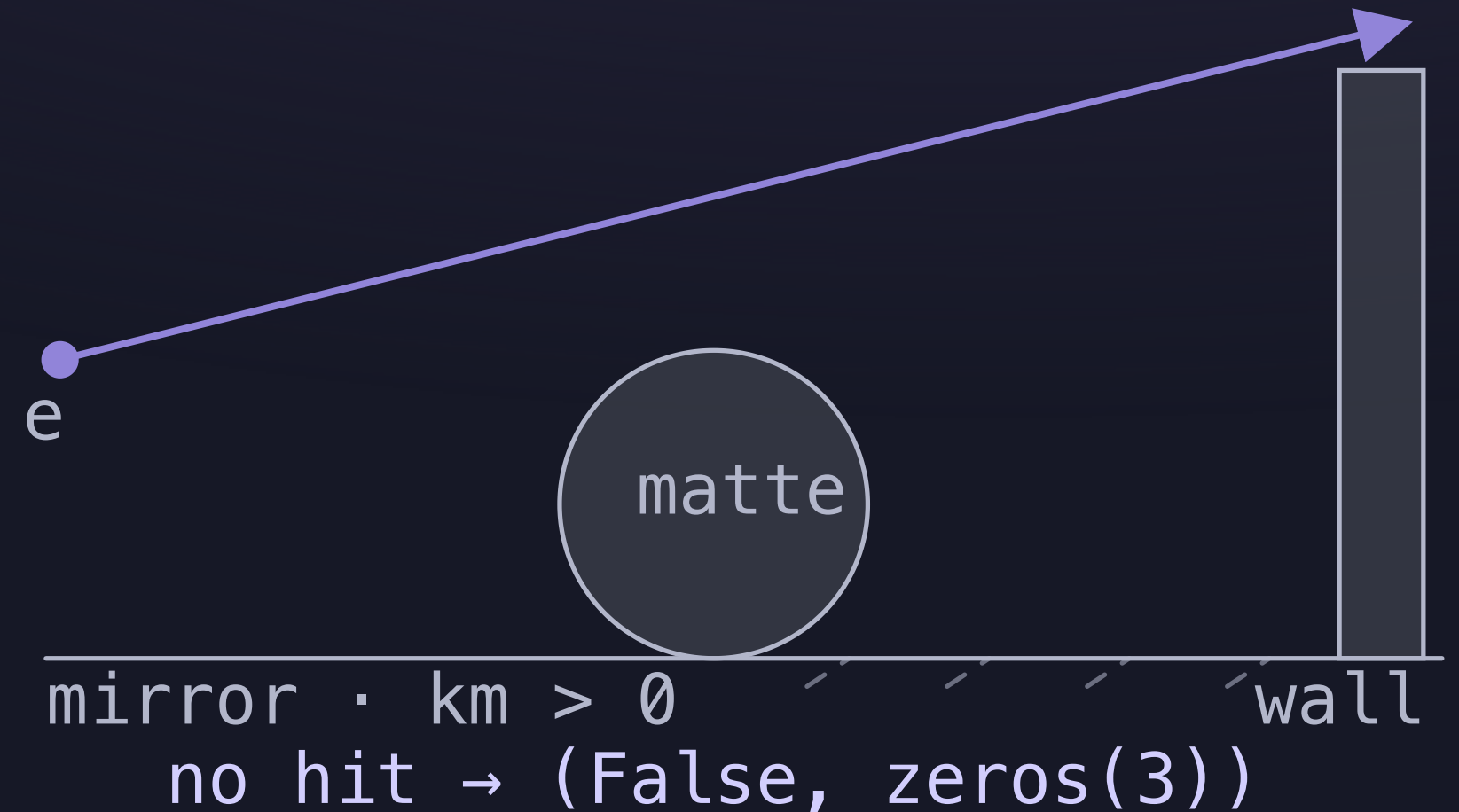
raycolor

YOUR TASK — 14 MARKS

- `raycolor(ray, min_t, objects, lights, num_recursive_calls) → (hit, rgb)`: shade the first hit, then add the mirror ray.

ALGORITHM

- 1 `first_hit`. Nothing hit $\rightarrow$ `(False, zeros(3))`: the ray escaped.
- 2 Shade the hit with `blinn_phong_shading` — for a matte surface, done.
- 3 Mirror-like (`max(km) > 0`) and under `MAX_RECURSIVE_CALLS` $\rightarrow$ reflect the direction about `n`.
- 4 Recurse from the hit point, offset `1e-6`, with `num_recursive_calls + 1`; add `km ×` what comes back.



The base case: no hit is not an error — the ray left the scene, and the backend paints the background.

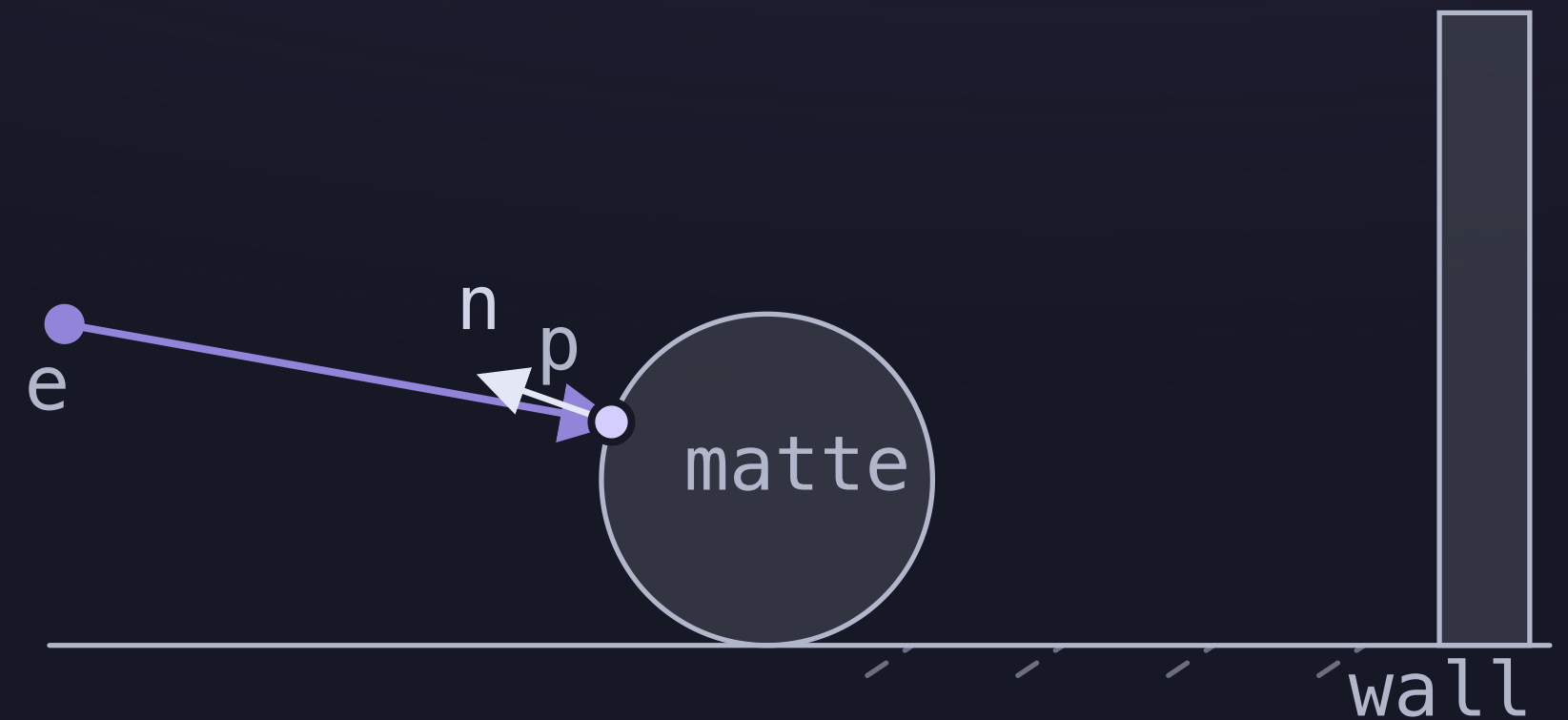
raycolor

YOUR TASK — 14 MARKS

- `raycolor(ray, min_t, objects, lights, num_recursive_calls) → (hit, rgb)`: shade the first hit, then add the mirror ray.

ALGORITHM

- 1 `first_hit`. Nothing hit $\rightarrow$ `(False, zeros(3))`: the ray escaped.
- 2 Shade the hit with `blinn_phong_shading` — for a matte surface, done.
- 3 Mirror-like ($\max(km) > 0$) and under `MAX_RECURSIVE_CALLS` $\rightarrow$ reflect the direction about n .
- 4 Recurse from the hit point, offset $1e-6$, with `num_recursive_calls + 1`; add $km \times$ what comes back.



`matte: shade  $\rightarrow$  (True, rgb)`

A matte material has $km = (0, 0, 0)$: the shaded colour *is* the answer. Most pixels in most scenes end here.

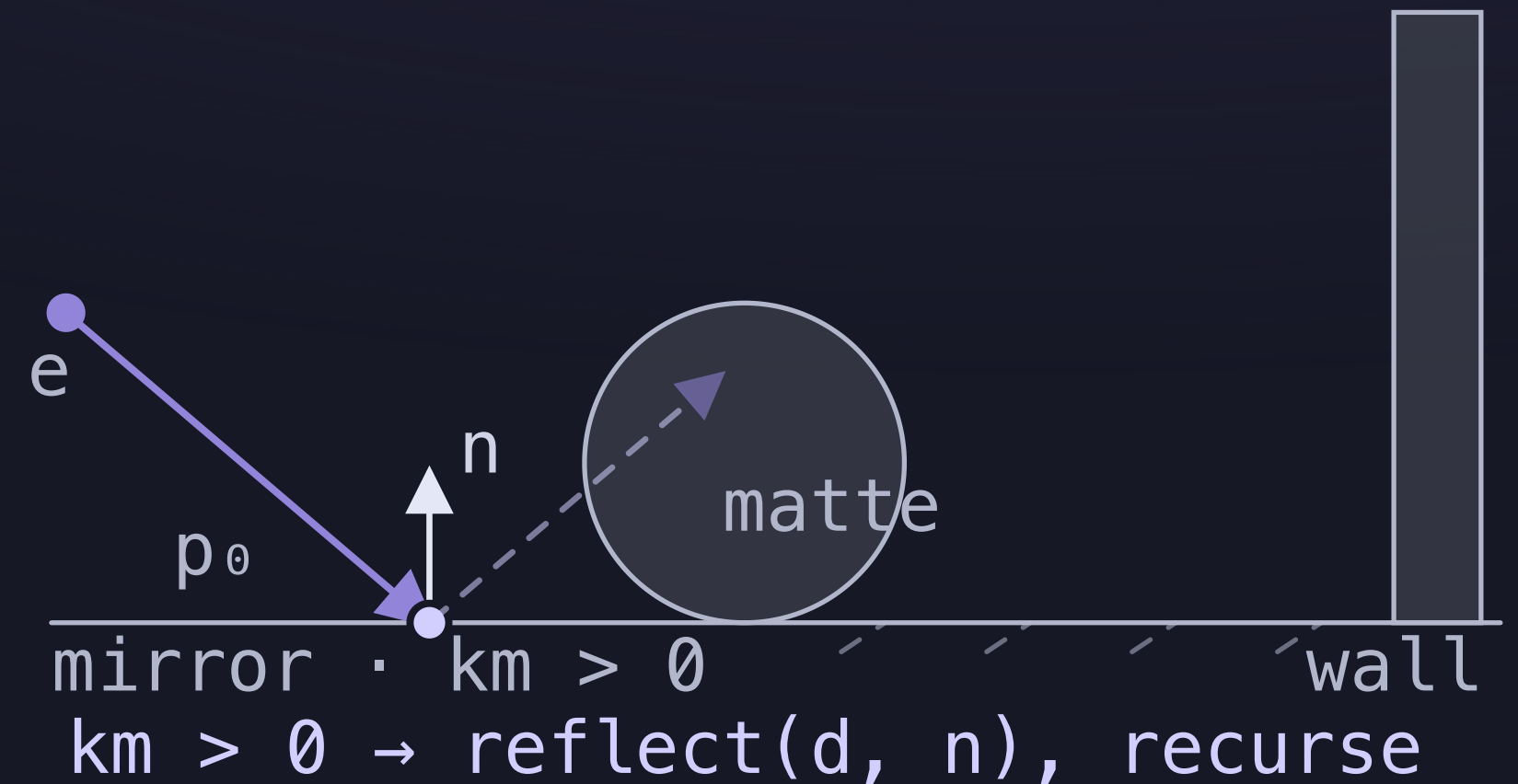
raycolor

YOUR TASK — 14 MARKS

- `raycolor(ray, min_t, objects, lights, num_recursive_calls) → (hit, rgb)`: shade the first hit, then add the mirror ray.

ALGORITHM

- 1 `first_hit`. Nothing hit $\rightarrow$ (`False`, `zeros(3)`): the ray escaped.
- 2 Shade the hit with `blinn_phong_shading` — for a matte surface, done.
- 3 Mirror-like (`max(km) > 0`) and under `MAX_RECURSIVE_CALLS` $\rightarrow$ reflect the direction about `n`.
- 4 Recurse from the hit point, offset `1e-6`, with `num_recursive_calls + 1`; add `km ×` what comes back.



Reflect about the **unit** normal; start the mirror ray `1e-6` along it — the same epsilon as the shadow rays.

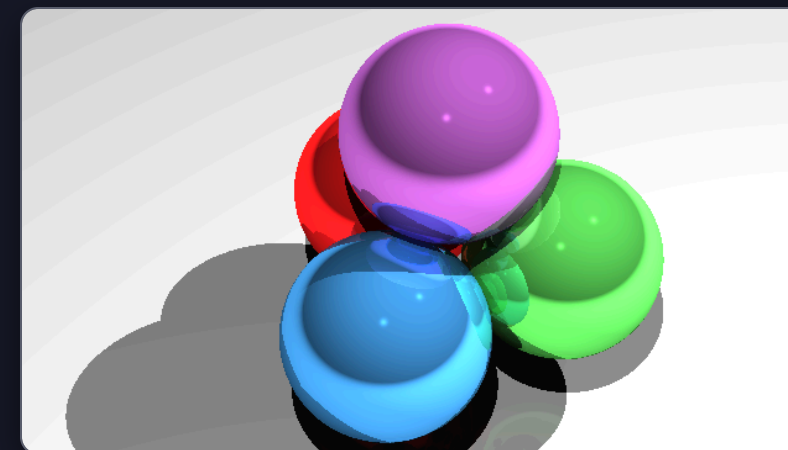
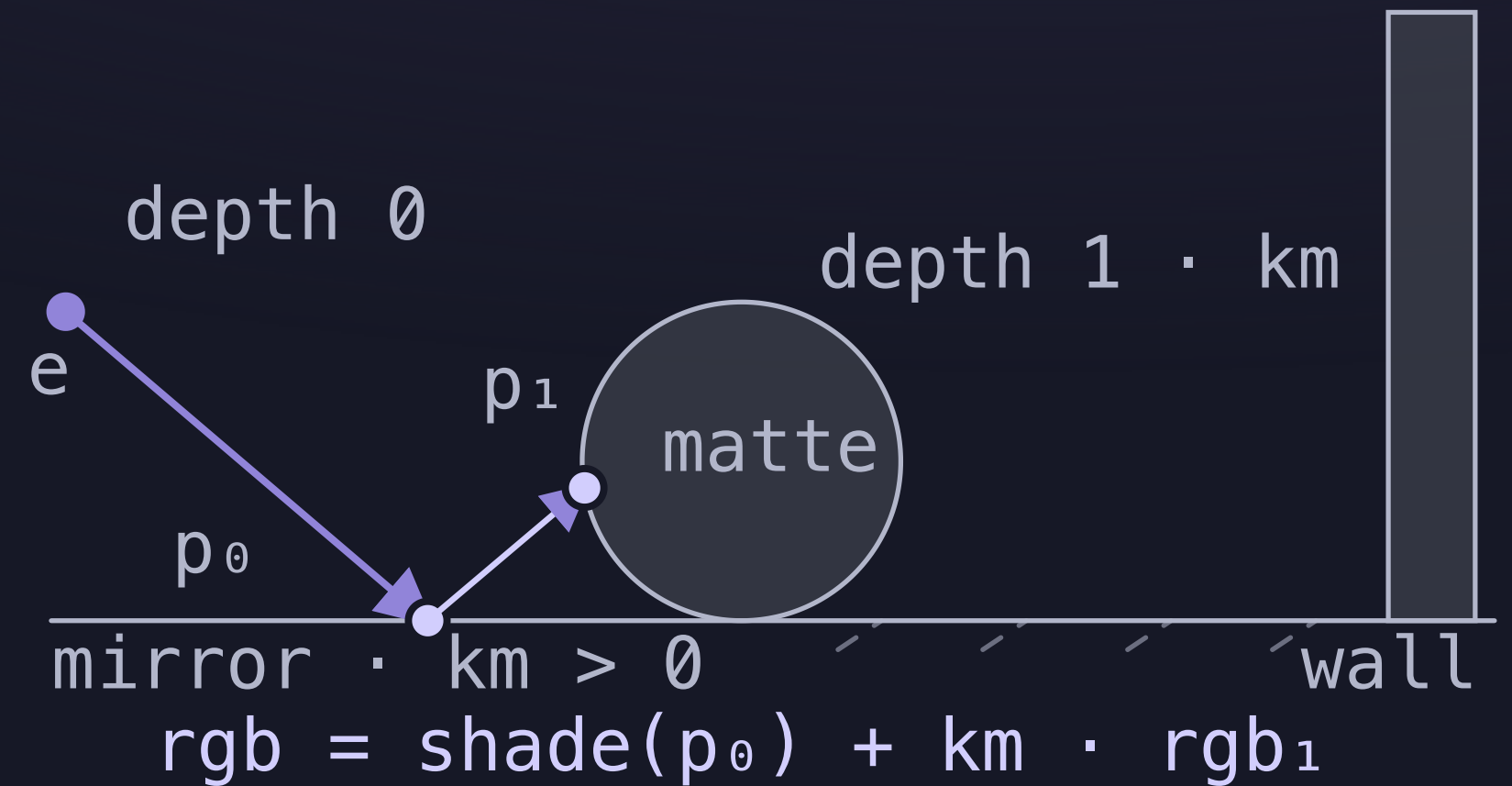
raycolor

YOUR TASK — 14 MARKS

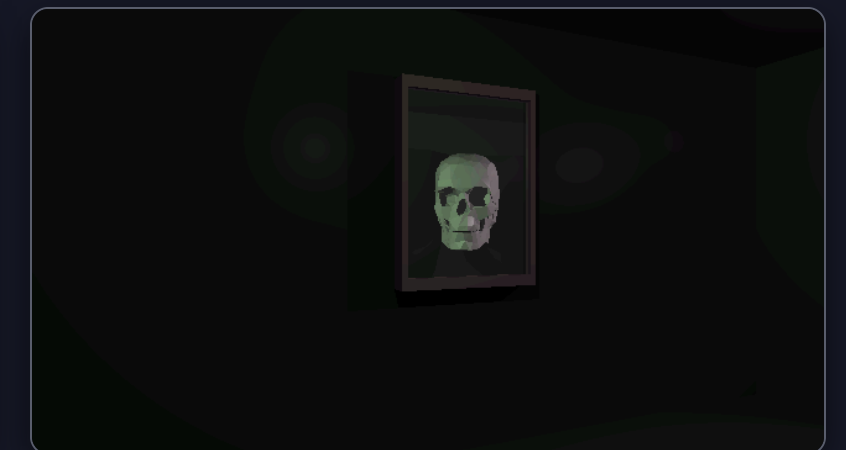
- `raycolor(ray, min_t, objects, lights, num_recursive_calls) → (hit, rgb)`: shade the first hit, then add the mirror ray.

ALGORITHM

- 1 `first_hit`. Nothing hit $\rightarrow$ (`False`, `zeros(3)`): the ray escaped.
- 2 Shade the hit with `blinn_phong_shading` — for a matte surface, done.
- 3 Mirror-like (`max(km) > 0`) and under `MAX_RECURSIVE_CALLS` $\rightarrow$ reflect the direction about n .
- 4 Recurse from the hit point, offset $1e-6$, with `num_recursive_calls + 1`; add `km ×` what comes back.



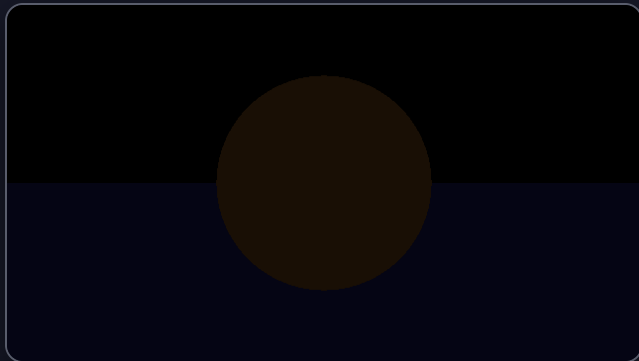
sphere-packing — needs all 10 bounces



mirror — a skull seen only in a mirror

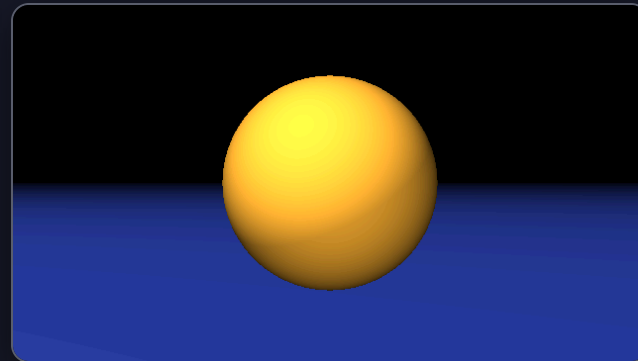
Debug one shading term at a time

```
$ python main.py --stages --height 112 # → out/<scene>.stages/1-ambient.png ... 5-reflection.png  
$ python main.py --stage shadows      # just one term
```



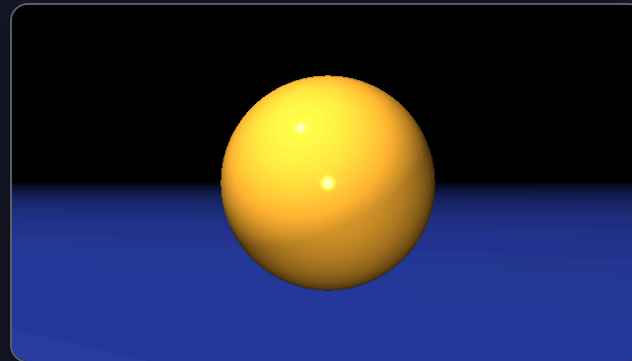
1 · ambient

a faint object-id image.
Wrong? → `viewing_ray`,
intersections, `first_hit`.
Try `--part 1`.



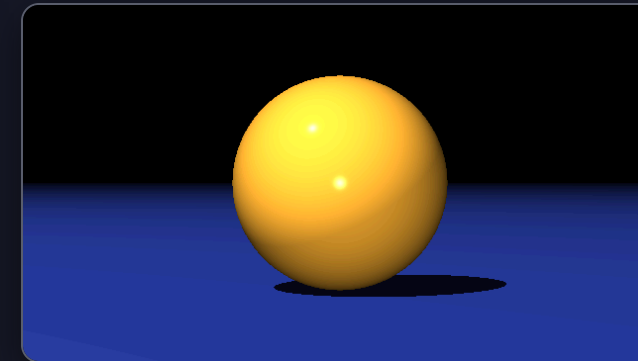
2 · + diffuse

flat, or lit from the wrong
side? → $n \cdot l$, your normals,
the light directions.



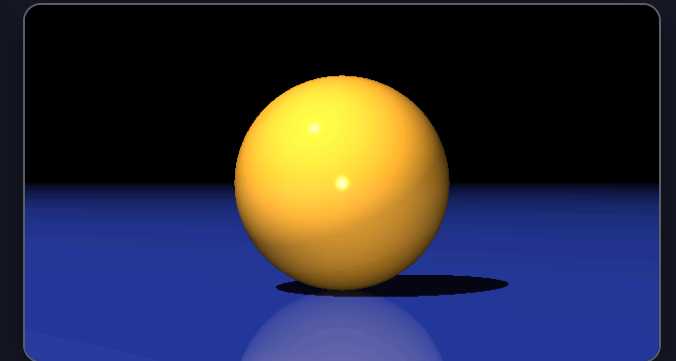
3 · + specular

no highlight, or a huge dull
one? → the half-vector h ,
or the exponent on the
wrong term.



4 · + shadows

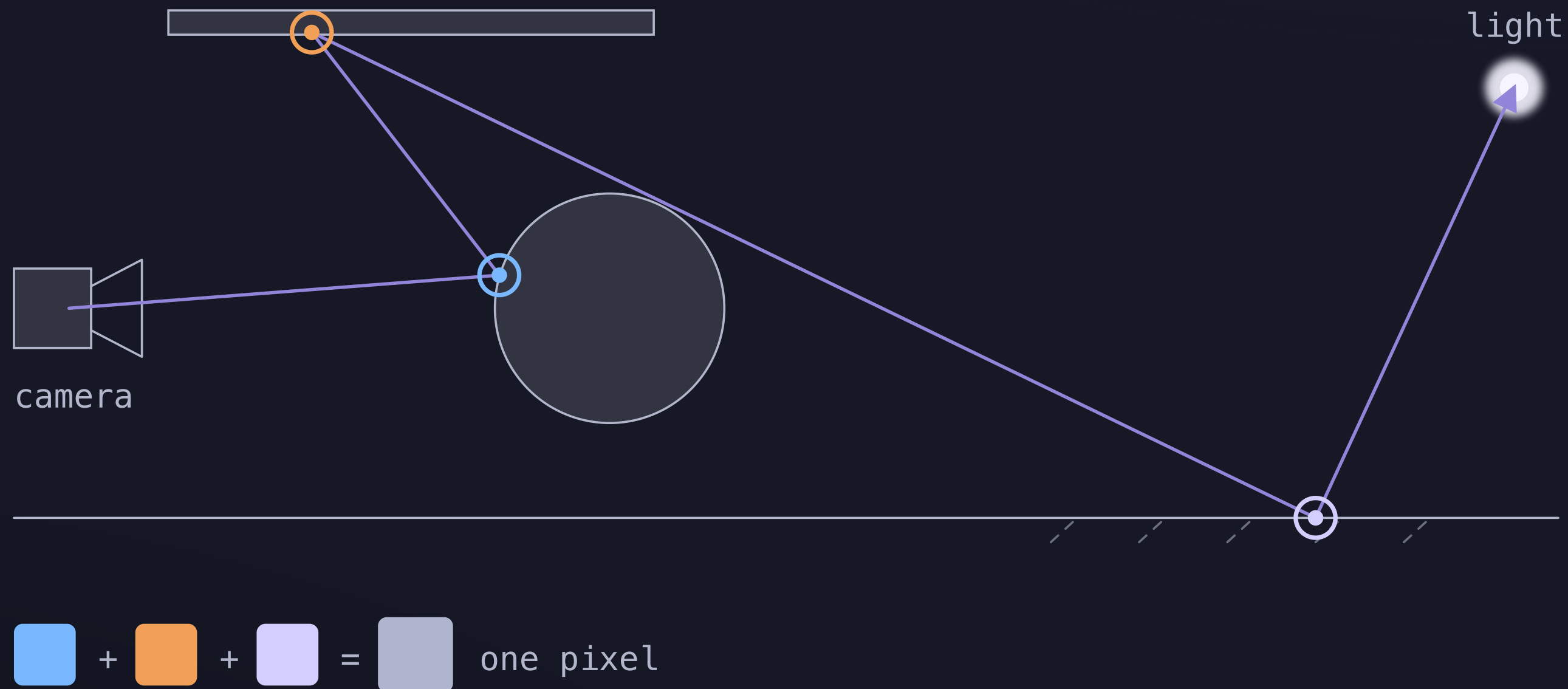
none, or speckled black
acne? → the shadow ray's
`epsilon`, or `t` vs `max_t`.



5 · + reflection

mirrors black, or blown out?
→ `reflect`, the `km`
weighting, the cap, or the
mirror ray's `epsilon`.

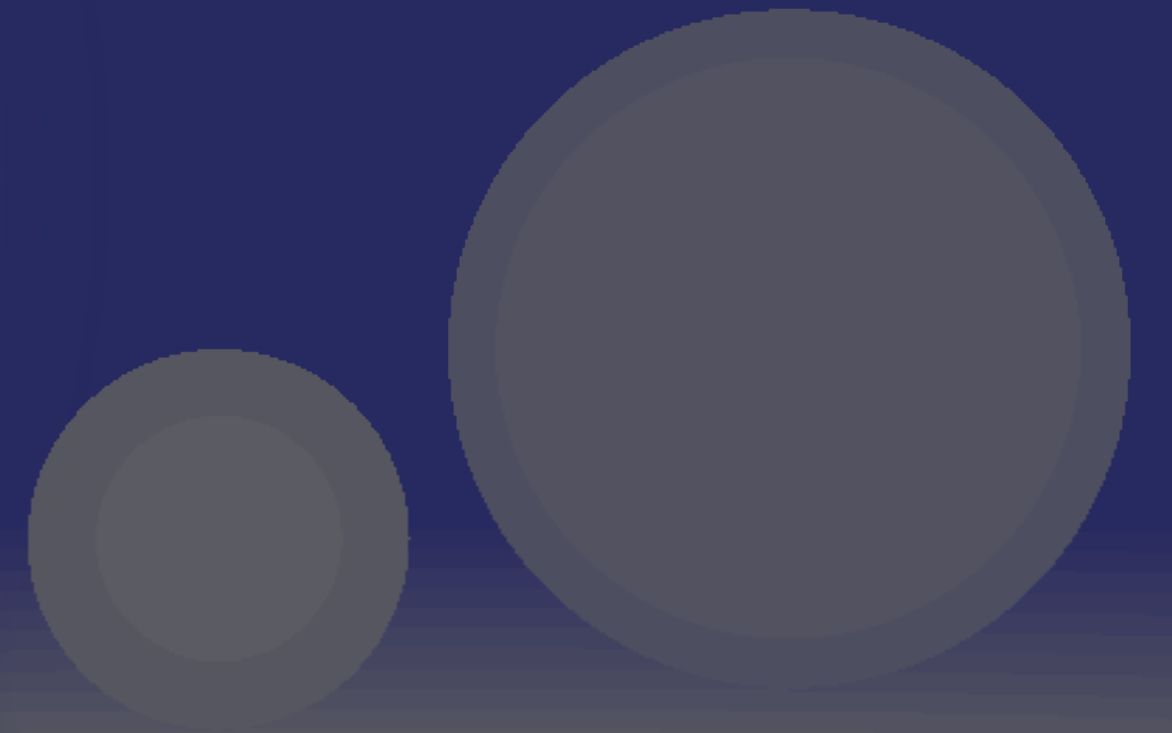
How the ten functions fit together



Every bounce runs the same three calls — `first_hit`, `light_direction`, `blinn_phong_shading` — and a mirror adds `reflect` and one more `raycolor`.

05

— Grading and debugging



How the mark is computed

Public tests — 30% of each function's marks

- The checks shipped in `tests/` — exactly what `run_tests.py` and `check_my_work.py` run.
- Geometry and lighting functions are compared numerically; `blinn_phong_shading` and `raycolor` by a small rendered image within tolerance.
- Every input and expected output is readable in `tests/expected/`, or load `tests/golden/a2_golden.npz` directly.

Private tests — 70% of each function's marks

- The same checks on different scenes: another sphere, a tilted plane, a triangle elsewhere.
 - They catch code that reproduces the published numbers without implementing the function.
 - A correct implementation needs nothing beyond the public data to pass them.
-
- Do not copy code from the internet or from AI tools. Read explanations (Wikipedia, the textbook) and cite your references in comments.

Submitting on MarkUs

Where

- markus.teach.cs.toronto.edu
- Under *Assignment 2: Ray Tracing*

What

- Every .py file in your src/ directory
 - all ten.
- Nothing from tests/, backend/ or cgcommon/.

When

- Friday October 9, 11:59 pm.
- After that, grace tokens are used automatically, 12 hours each.

```
$ python check_my_work.py      # aim for Validated 10/10 before you submit
```

Reading a failing test

A numeric check is off

- Print the inputs from `tests/expected/` and work one case by hand — a sphere and an axis-aligned ray take a minute.
- Usual suspects: the far root, `min_t`, the `+ 0.5` pixel centre, the sign of `v`, a normalised direction that should not be.

The image is slightly off everywhere

- A coefficient: the `0.1` ambient, `I` missing from a term, the exponent on the wrong dot product.
- Or a normalised vector that is not unit — check `h`.

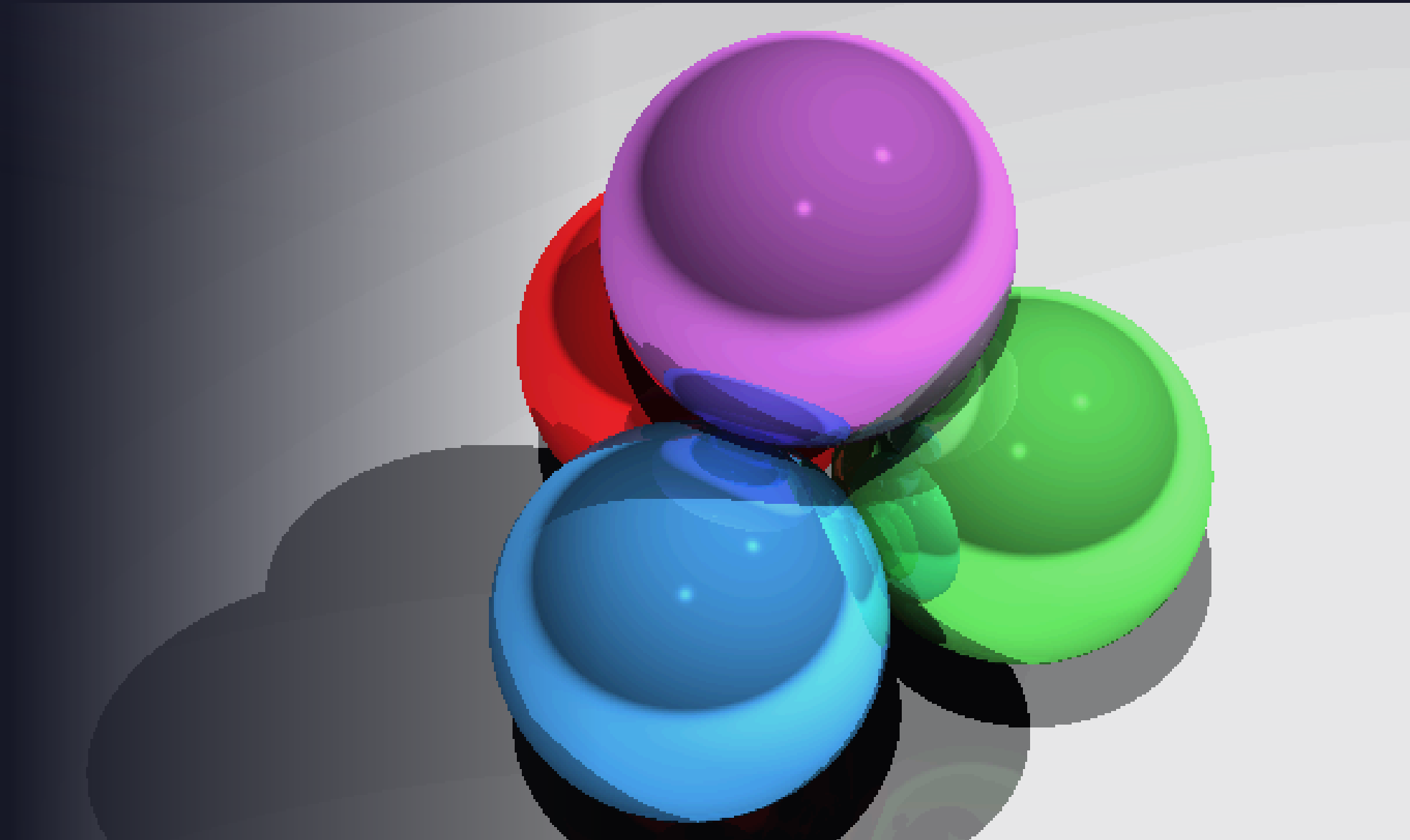
A region is wrong

- Black speckles or black surfaces: the `epsilon` on shadow or mirror rays.
- Dark mirrors: the recursion cap, `num_recursive_calls` not incremented, or `km` not applied.

```
out/test_diffs/<function>.got.png      # what your code produced
out/test_diffs/<function>.expected.png  # what was expected
out/test_diffs/<function>.diff.png      # where they differ (brighter = worse)
```

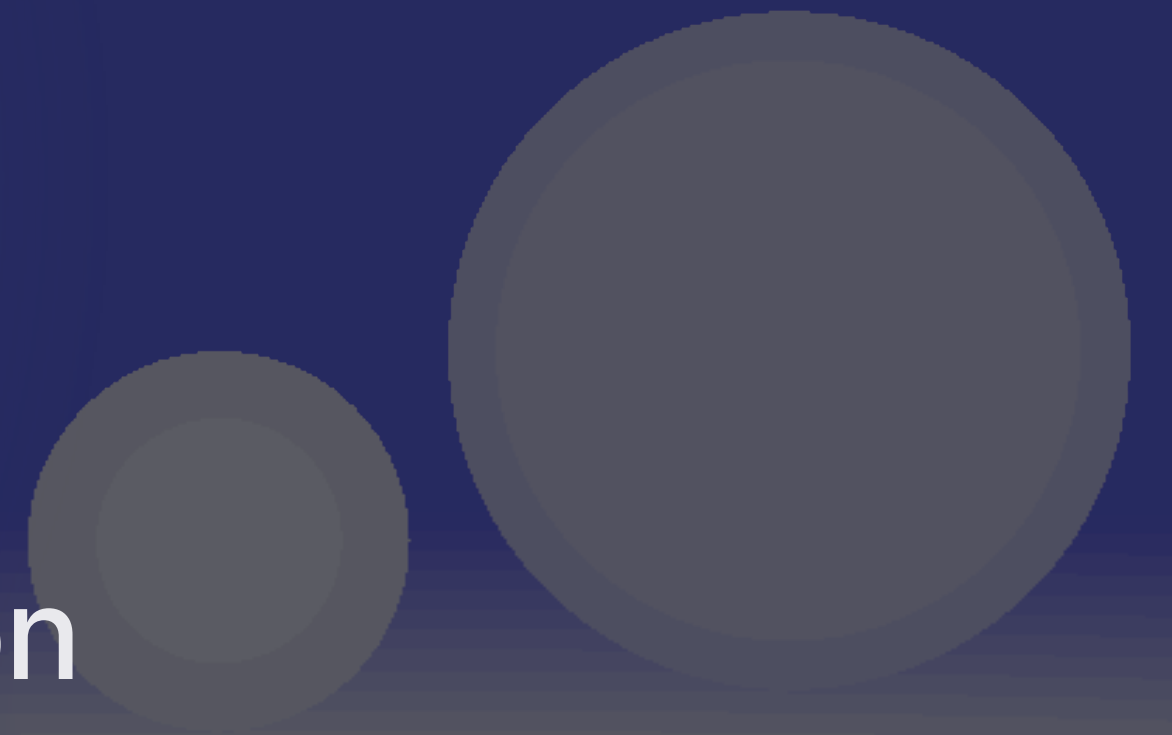
Questions?

A2 GitHub Issues ·
csc317tas@cs.toronto.edu



06

- Extra: formalizing the rendering equation



One equation for all of it

$$\boxed{L_o(p, \omega_o)} = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$

radiance leaving p toward the viewer

One equation for all of it

$$L_o(p, \omega_o) = \boxed{L_e(p, \omega_o)} + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$ radiance leaving p toward the viewer

$L_e(p, \omega_o)$ light the surface emits by itself

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) d\omega_i$$

$L_o(p, \omega_o)$

radiance leaving p toward the viewer

$L_e(p, \omega_o)$

light the surface emits by itself

$\int_{\Omega} \dots d\omega_i$

over every incoming direction on the hemisphere above p

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} \boxed{f_r(p, \omega_i, \omega_o)} \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$

radiance leaving p toward the viewer

$L_e(p, \omega_o)$

light the surface emits by itself

$\int_{\Omega} \dots d\omega_i$

over every incoming direction on the hemisphere above p

$f_r(p, \omega_i, \omega_o)$

the BRDF — of what arrives from ω_i , how much leaves toward ω_o

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot \boxed{L_i(p, \omega_i)} \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$

radiance leaving p toward the viewer

$L_e(p, \omega_o)$

light the surface emits by itself

$\int_{\Omega} \dots d\omega_i$

over every incoming direction on the hemisphere above p

$f_r(p, \omega_i, \omega_o)$

the BRDF — of what arrives from ω_i , how much leaves toward ω_o

$L_i(p, \omega_i)$

radiance arriving from ω_i — some other surface radiance out

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$

radiance leaving p toward the viewer

$L_e(p, \omega_o)$

light the surface emits by itself

$\int_{\Omega} \dots \, d\omega_i$

over every incoming direction on the hemisphere above p

$f_r(p, \omega_i, \omega_o)$

the BRDF — of what arrives from ω_i , how much leaves toward ω_o

$L_i(p, \omega_i)$

radiance arriving from ω_i — some other surface radiance out

$(n \cdot \omega_i)$

cosine foreshortening — how the surface tilts to that direction

The same equation, as your code

$$\boxed{L_o(p, \omega_o)} = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

$L_o(p, \omega_o)$

what raycolor returns

The same equation, as your code

$$L_o(p, \omega_o) = \boxed{L_e(p, \omega_o)} + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

`L += 0.1 * ka`

$L_o(p, \omega_o)$

what raycolor returns

$L_e(p, \omega_o)$

zero — no emitters in A2; a constant ambient stands in for the light we never trace

The same equation, as your code

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) d\omega_i$$

`L += Σlights [ kd · I · max(0, n·l) + ks · I · max(0, n·h)p ] + km · raycolor(r)`

$L_o(p, \omega_o)$

what raycolor returns

$L_e(p, \omega_o)$

zero — no emitters in A2; a constant ambient stands in for the light we never trace

$\int_{\Omega} \dots d\omega_i$

a finite sum: one term per light, plus one mirror direction

The same equation, as your code

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i$$

I · km · raycolor(reflect(d, n))

$L_o(p, \omega_o)$

what raycolor returns

$L_e(p, \omega_o)$

zero — no emitters in A2; a constant ambient stands in for the light we never trace

$\int_{\Omega} \dots \, d\omega_i$

a finite sum: one term per light, plus one mirror direction

$L_i(p, \omega_i)$

I for a light the shadow ray can see, or raycolor one bounce deeper

The same equation, as your code

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot \boxed{(n \cdot \omega_i)} \, d\omega_i$$

$\max(0, n \cdot l)$

$L_o(p, \omega_o)$

what raycolor returns

$L_e(p, \omega_o)$

zero — no emitters in A2; a constant ambient stands in for the light we never trace

$\int_{\Omega} \dots d\omega_i$

a finite sum: one term per light, plus one mirror direction

$L_i(p, \omega_i)$

I for a light the shadow ray can see, or raycolor one bounce deeper

$(n \cdot \omega_i)$

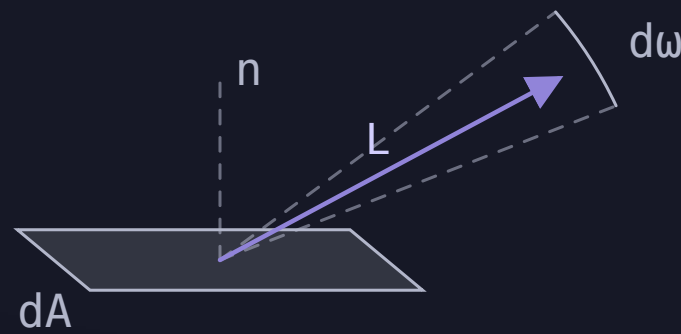
exactly $\max(0, n \cdot l)$

What the units are

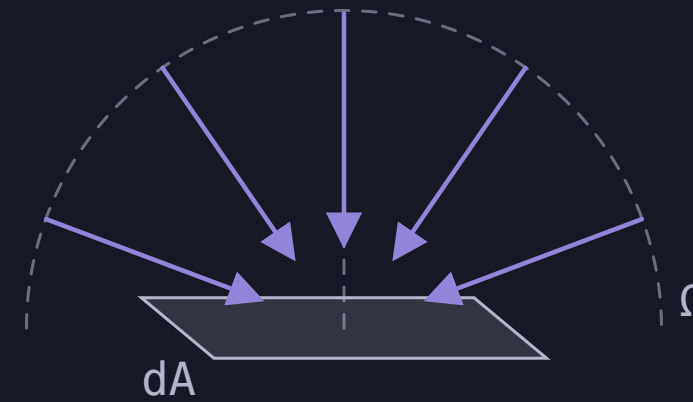
$$\boxed{L_o(p, \omega_o)} = \boxed{L_e(p, \omega_o)} + \int_{\Omega} f_r(p, \omega_i, \omega_o) \cdot \boxed{L_i(p, \omega_i)} \cdot (n \cdot \omega_i) \, d\omega_i$$

$\boxed{}$ radiance, $\text{W}/(\text{m}^2 \cdot \text{sr})$

$\int_{\Omega} \boxed{L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, d\omega_i}$ irradiance, W/m^2



Radiance — one patch, one direction: $\text{W}/(\text{m}^2 \cdot \text{sr})$.



Irradiance — every direction at once: W/m^2 . So f_r is $1/\text{sr}$.

Questions?

A2 GitHub Issues ·
csc317tas@cs.toronto.edu

